

Fredrik Hammarberg

Optimising Weak Reference Processing in the JVM Z Garbage Collector

*Reference Pipeline Optimisations and an Exploratory
Weak-Field Mechanism for the JVM*



UPPSALA
UNIVERSITET

Abstract

Reference processing can be a critical component of Java garbage collection latency, particularly in workloads with large numbers of weakly reachable objects. This thesis investigates how the Z Garbage Collector (ZGC) can reduce overhead when processing `WeakReference` objects that are not associated with a `ReferenceQueue`. Three orthogonal implementation strategies are developed in an OpenJDK research fork: a queue-less fast path that skips enqueue handling, a cache-friendlier dynamic-array representation for discovered references, and an optimised clear path that removes unnecessary per-reference work.

To evaluate these changes, a controlled benchmarking framework was constructed with repeated runs, CPU isolation, memory monitoring, and GC-log analysis. The evaluation compares all combinations of the three optimisation strategies against a baseline configuration using both single-object stress and multi-object phased-release workloads.

In addition to `WeakReference`-based optimisations, the thesis introduces an exploratory weak-field mechanism via a Java field annotation and corresponding VM support. This allows a representation-level comparison between object-based weak references and field-based weak semantics under the same experimental setup.

The results show that the strongest `WeakReference` variants are those that combine the specialised clear path with the dynamic-array representation. In the single-object stress benchmark, these variants reduce the targeted reference-processing subphases by more than 70% and reduce major-collection time by up to 2.9%. In the multi-object benchmark, subphase improvements remain substantial, but end-to-end major-collection gains are small. The exploratory weak-field mechanism performs strongly in the single-object benchmark but regresses in the multi-object benchmark, indicating that field-based weak semantics are promising but not yet a general replacement for optimised `WeakReference` processing.

Acknowledgments

I want to express my sincere gratitude to my supervisor for his support throughout this thesis work. It is through his help and expertise that I was able to learn so much about the JVM and complete this thesis.

I also want to thank my subject reviewer for his academic guidance and overall feedback, which helped strengthen the structure and quality of this thesis.

Additionally, I want to thank the Uppsala Multidisciplinary Center for Advanced Computational Science at Uppsala University for providing access to the resources necessary to conduct the performance evaluations and experiments described in this thesis.

Lastly, I want to thank my colleagues at Oracle for their insights and feedback on my work throughout the development process. Without their help, I would not have been able to achieve the results presented in this thesis.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

Acknowledgments	iii
1 Introduction	1
1.1 Motivation	2
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Scope and Delimitations	3
2 Background	4
2.1 Garbage Collection in the JVM	4
2.1.1 Garbage Collection Phases	5
2.1.2 Garbage Collection Barriers	5
2.1.3 Generational Collection	6
2.2 Java Reference Types	6
2.2.1 The Reference Object Lifecycle	7
2.2.2 WeakReference and ReferenceQueue	8
2.3 The JVM Execution Model	9
2.4 The Z Garbage Collector	9
2.4.1 Coloured Pointers and Barriers	10
2.4.2 Concurrent Phases and Parallel Workers	10
2.4.3 Access Decorators	11
2.5 Reference Processing Pipeline in ZGC	11
2.5.1 Discovery	12
2.5.2 Processing	13
2.5.3 Enqueue	14
3 Design and Implementation	15
3.1 Skipping Enqueue Path	15
3.1.1 Implementation of Skipping Enqueue Path	15
3.2 Dynamic Array for Discovered References	16
3.2.1 Implementation of Dynamic Array for Discovered References	17
3.3 Optimised Clear Path	17
3.3.1 Implementation of Optimised Clear Path	19
3.4 Weak Fields	19
3.4.1 Implementation of Weak Fields	20
4 Evaluation	25
4.1 Optimisation Variants	25

4.2	Performance Evaluation	25
4.2.1	Benchmark Design	26
4.2.2	Execution Environment	27
4.2.3	Measurement Methodology	28
4.3	Presentation of Results	29
5	Analysis of Results	43
5.1	Non-Strong Reference Processing	43
5.1.1	Effect of the Clear Path and Dynamic Array	43
5.1.2	Effect of the Skip-Enqueue Separation Alone	44
5.1.3	Comparison with Weak Fields	44
5.2	Overall GC Collection Times	44
5.2.1	WeakReference Variants	44
5.2.2	Violin Plot Shape and the Bimodal Pattern	45
5.2.3	Single versus Multi-Object Benchmark	45
5.3	Inter-Model Comparison: Weak Fields versus WeakReference ..	45
5.4	Memory Behaviour	46
5.4.1	Auxiliary Memory Usage and the Dynamic Array	46
5.4.2	Java Heap Usage	46
6	Related Work	47
6.1	Go	47
6.2	Python	47
6.3	Synthesis	48
7	Discussion	49
7.1	Interpretation of Results	49
7.2	Future Work	50
8	Conclusion	51
	References	52
	Appendix A: Usage of Generative Artificial Intelligence Tools	54
A.1	Use During Implementation	54
A.2	Use for Tests and Scripts	54
A.3	Use During Writing	54
	Appendix B: Source Code	55
B.1	Dynamic-array Data Structure	55
B.2	Reference Processor	58
B.2.1	Interface	58
B.2.2	Implementation Excerpts	61
B.3	Weak Fields	66
B.3.1	Annotation	66
B.3.2	Data Structure	67
B.3.3	Reference Processor	71

B.3.4	Marking Closure	76
B.3.5	Compiler and Interpreter Support	77
Appendix C:	Per-Variant Patch Details	84
C.1	all	84
C.2	clear_path_dyn	84
C.3	clear_path_only	85
C.4	clear_path_sep	85
C.5	dyn_only	86
C.6	sep_dyn	86
C.7	sep_only	87
C.8	weak_fields	87
Appendix D:	Summary Statistics	89
D.1	Non-Strong Processing Time	89
D.2	Major Collection Time	89
D.3	Old-Generation Collection Time	89
D.4	Concurrent Mark Follow Time	89
D.5	Auxiliary GC Memory	89
D.6	Java Heap	89
D.7	Total Process RSS	89
Appendix E:	Additional Benchmark Plots	93

List of Tables

Table 4.1: Optimisation variants	26
Table 4.2: Patch sizes per variant	26
Table 4.3: Cluster node hardware specification	27
Table 4.4: JVM options used in all benchmark runs	28
Table 4.5: Continuous monitoring metrics in the measurement pipeline ..	29
Table 4.6: GC log metrics in the measurement pipeline	30
Table 8.9: Summary statistics for Non-Strong Processing across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).	90
Table 8.10: Summary statistics for Major Collection across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).	90
Table 8.11: Summary statistics for Old Generation across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).	91
Table 8.12: Summary statistics for Concurrent Mark Follow across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).	91
Table 8.13: Summary statistics for Aux Memory per-run maximum across all variants. Values in MB ($n = 100$ per variant).	92
Table 8.14: Summary statistics for Java Heap per-run maximum across all variants. Values in MB ($n = 100$ per variant).	92
Table 8.15: Summary statistics for RSS per-run maximum across all variants. Values in MB ($n = 100$ per variant).	92

List of Figures

Figure 2.1:	Reference lifecycle states	8
Figure 2.2:	ZGC reference-processing pipeline	12
Figure 2.3:	Cross-generational reference case	13
Figure 4.1:	Major Collection timing (single-object)	31
Figure 4.2:	Major Collection timing (multi-object)	32
Figure 4.3:	Old Generation timing (single-object)	33
Figure 4.4:	Old Generation timing (multi-object)	34
Figure 4.5:	Young Generation (Promote All) timing (single-object)	35
Figure 4.6:	Young Generation (Promote All) timing (multi-object)	36
Figure 4.7:	Old Phase: Concurrent Mark timing (single-object)	37
Figure 4.8:	Old Phase: Concurrent Mark timing (multi-object)	38
Figure 4.9:	Old Phase: Concurrent Process Non-Strong timing (single-object)	39
Figure 4.10:	Old Phase: Concurrent Process Non-Strong timing (multi-object)	40
Figure 4.11:	Memory median percentage differences (single-object)	41
Figure 4.12:	Memory median percentage differences (multi-object)	42
Figure 8.1:	Old Phase: Concurrent Relocate timing (single-object)	94
Figure 8.2:	Old Phase: Concurrent Relocate timing (multi-object)	95
Figure 8.3:	Young Phase: Concurrent Mark timing (single-object)	96
Figure 8.4:	Young Phase: Concurrent Mark timing (multi-object)	97
Figure 8.5:	Young Phase: Concurrent Relocate timing (single-object)	98
Figure 8.6:	Young Phase: Concurrent Relocate timing (multi-object)	99
Figure 8.7:	Memory per-run maximum distributions (single-object)	100
Figure 8.8:	Memory per-run maximum distributions (multi-object)	101

List of Listings

Listing 2.1:	WeakReference constructors	8
Listing 3.1:	Queue-less weak discovery split	16
Listing 3.2:	Dynamic-array weak processing	18
Listing 3.3:	Optimised-clear-path fast inactive check	22
Listing 3.4:	Weak-field annotation	22
Listing 3.5:	Weak-field discovery	23
Listing 3.6:	Weak-field processing	24

1. Introduction

In managed runtimes, garbage collection (GC) is a central part of application performance: GC pauses affect latency, while concurrent GC work competes with application threads for processor time and memory bandwidth [1, 2]. OpenJDK’s Z Garbage Collector (ZGC) is designed for this regime, combining millisecond-scale pauses with a predominantly concurrent architecture intended to keep pause times largely independent of heap size [3]. Since its introduction as an experimental collector, ZGC has matured into a production-ready, generational collector, making it a suitable platform for studying the overhead of individual components in low-latency garbage collection [4, 5].

One aspect of ZGC that has received comparatively little optimisation attention is the processing of *weak references*. In Java, the class `WeakReference` provides weak semantics, allowing applications to retain references to objects without preventing those objects from being collected [6, 7]. When the garbage collector determines that an object is only reachable through a `WeakReference`, i.e. no longer strongly reachable, it clears the reference and, if the reference is associated with a `ReferenceQueue`, enqueues it for application-level processing [8, 9]. This enqueueing step serves as a garbage collector callback, notifying the application that the `WeakReference` has been cleared. Weak semantics, both with and without such callbacks, are used in a range of programming patterns and data structures, for example the Java library `WeakHashMap` [10].

The current ZGC reference-processing pipeline treats all weak references uniformly: discovered references are linked into an intrusive linked list, processed, and then handed to a Java-level daemon thread (the `ReferenceHandler`) for enqueueing [11, 8]. This design imposes disproportionate overhead on weak references that are not registered with a `ReferenceQueue`. Such references are used solely for weak semantics and therefore do not require the callback mechanism provided by the queue. Nevertheless, the `WeakReference` object still needs to go through the entire processing pipeline, even though the application has no need for enqueueing.

This inefficiency has previously been identified in the OpenJDK bug tracker [12], which notes the unnecessary traversal of the pending list for references without a `ReferenceQueue`. However, as of April 2026, no implementation addressing this issue has been integrated into the JDK. Building on that observation, this thesis investigates whether targeted modifications to ZGC’s reference-processing pipeline can reduce reference-processing overhead. It explores three orthogonal optimisation

approaches for `WeakReference`-based processing: skipping the enqueue path for references without queues, replacing the intrusive linked list with a dynamic array, and optimising the codepath for clearing references. In addition, it examines a weak-field mechanism as an alternative implementation of weak semantics that avoids `WeakReference` objects entirely.

1.1 Motivation

Weak references, unlike regular references, require explicit processing by the JVM garbage collector [11, 13]. Each discovered weak reference must be individually evaluated, its referent field cleared if the referent is unreachable, and the reference potentially enqueued. This per-reference processing cost scales linearly with the number of weak references, making it a bottleneck in applications that allocate many weak references.

The problem is amplified by a mismatch between the API design and its implementation. Java’s `WeakReference` API provides an optional `ReferenceQueue` parameter [7], allowing applications to choose whether they want the callback mechanism or not. However, ZGC’s reference processor does not take advantage of this distinction, treating all weak references as if they require enqueueing. This results in unnecessary work for references that do not use a `ReferenceQueue`. At the same time, weak semantics are currently implemented through separate `WeakReference` objects to support this callback mechanism. Applications therefore also pay the allocation and processing cost of those objects even when they only need weak semantics.

ZGC’s concurrent architecture makes both of these inefficiencies particularly relevant. Because ZGC processes references concurrently with the application [3, 11], any reduction in per-reference overhead translates into better application performance. This motivates two complementary lines of investigation: optimising the existing `WeakReference` pipeline for the queue-less case, and exploring whether weak semantics expressed through annotated object fields can reduce overhead further by avoiding `WeakReference` objects altogether.

1.2 Problem Statement

The current ZGC reference processing pipeline does not distinguish between weak references with and without a `ReferenceQueue` during its phases [11]. All cleared references are threaded onto a pending list, transferred to the `ReferenceHandler` thread, and iterated regardless of whether they will actually be enqueued or not [8]. Additionally, the intrusive linked-list data structure used for discovered references exhibits poor cache locality [14]. Beyond these pipeline inefficiencies, representing weak semantics through separate

`WeakReference` objects introduces object and heap overhead that may be avoidable if weak semantics is instead expressed directly through annotated object fields.

1.3 Research Questions

This thesis addresses the following research questions:

- RQ1** How do modifications to ZGC’s processing of `WeakReference` objects without a `ReferenceQueue` affect the wall-clock time of reference processing, the total GC cycle time, the GC memory footprint, and Java heap usage in select workloads?
- RQ2** How does expressing weak semantics through annotated object fields affect reference processing time, GC cycle time, GC memory footprint, and Java heap usage compared to the optimised `WeakReference`-based approach of RQ1?

1.4 Scope and Delimitations

This thesis focuses on the reference processing pipeline within ZGC, specifically targeting weak references without a `ReferenceQueue`. The primary implementation work concerns `WeakReference`-based processing, with weak fields treated as an extension evaluated in the same benchmarking framework. Although the optimisation approaches are described in general terms, the implementation and evaluation remain ZGC-specific. Furthermore, some of the optimisations, such as the optimised clear path, may have broader applicability beyond weak references without a `ReferenceQueue`. However, those broader applications fall outside the scope of this thesis and are not evaluated.

The correctness of the optimisations is only verified through the standard OpenJDK test suite and manual inspection. Performance measurements are conducted on a single hardware platform under controlled conditions and therefore may not generalise to all environments.

2. Background

To answer the research questions posed in Section 1.3, it is necessary to understand key concepts of the Java Virtual Machine’s (JVM) garbage collection mechanisms, the design and implementation of the Z Garbage Collector (ZGC), the Java reference types and their processing pipeline within ZGC, and general principles of garbage collection.

2.1 Garbage Collection in the JVM

The JVM provides automatic memory management through garbage collection (GC). Rather than requiring the programmer to explicitly allocate and free memory as in languages such as C or C++, the JVM tracks which objects are reachable from the application’s root set and reclaims the memory of objects that are no longer reachable. The root set consists of all data that is directly accessible to the application, such as local variables on the stack, static fields, and active threads.

The JVM ships with several garbage collector implementations, each designed with different trade-offs between throughput, latency, and memory footprint [1]. The collectors are:

- **Serial GC** A single-threaded collector that performs all garbage collection work using one thread, avoiding inter-thread communication overhead. It is best suited to single-processor systems and small heaps.
- **Parallel GC** A throughput-oriented generational collector that uses multiple GC threads to speed up collection on multiprocessor systems. It is intended for medium to large data sets.
- **G1 GC (Garbage-First)** A mostly concurrent, region-based collector designed to scale from small to large multiprocessor machines. It targets predictable pause times with high probability while maintaining high throughput and is the default collector on most platforms [1].
- **ZGC (Z Garbage Collector)** A scalable, low-latency collector that performs expensive GC work concurrently with application threads. It targets pause times of only a few milliseconds (largely independent of heap size) and supports heap sizes from 8 MB to 16 TB.

This thesis focuses on ZGC, as its low-latency design and concurrent reference processing pipeline can particularly benefit from optimisations that reduce the amount of work done during reference processing.

2.1.1 Garbage Collection Phases

Garbage collection generally occurs multiple times (cycles) during the lifetime of a Java application. Each cycle consists of several distinct phases, each with specific responsibilities. The exact phases and their order can vary between collectors, but a common structure is as follows:

1. **Mark Phase** The collector identifies all live objects by traversing from the root set and marking all reachable objects. Dead objects are those that are not marked. This phase may be performed concurrently with the application or during a stop-the-world pause, depending on the collector design.
2. **Reference Processing Phase** After marking completes, the collector processes *Reference* objects discovered during the mark phase. This phase determines which references have dead referents and prepares them for enqueueing. Like marking, this phase may be concurrent or done during a pause.
3. **Sweep/Reclamation Phase** The collector reclaims memory occupied by dead objects. Some collectors use a sweep algorithm (scanning the heap and freeing unmarked objects) whilst others use evacuation (copying live objects to new memory regions).
4. **Compaction/Relocation Phase** Some collectors compact the heap to eliminate fragmentation, moving live objects to consolidate free space. Others relocate objects incrementally as part of the collection process.

The order and concurrency model of these phases varies significantly between collectors. Serial and Parallel GC typically perform all phases in stop-the-world pauses. G1 performs marking and sweeping mostly concurrently with short pauses for synchronisation. ZGC performs marking, reference processing, and relocation concurrently, using barriers to maintain consistency with concurrent application threads.

2.1.2 Garbage Collection Barriers

In JVM garbage collectors, a *barrier* is a small piece of code inserted around memory accesses so that the collector can maintain correctness while the application is running [2]. The term is slightly misleading: barriers are not global locks and do not stop threads. Instead, they are local checks or bookkeeping actions attached to individual loads and stores of object references [2, 3, 15].

Barriers are especially important in concurrent collectors because GC and application threads observe and modify heap state at the same time. Rather than pausing the application for long periods, the collector uses barriers to spread consistency work across ordinary execution [3, 2].

Two broad classes are common:

- **Load (read) barriers**, executed when a reference is read. These can validate or transform the loaded pointer before it is used by the application [3, 15].
- **Store (write) barriers**, executed when a reference field is overwritten. These record metadata needed by the collector, for example to preserve marking invariants or to track inter-generational pointers [2, 5].

2.1.3 Generational Collection

In current ORacle JDK releases, all four supported JVM garbage collectors (Serial, Parallel, G1, and Z) use *generational collection*, based on the *weak generational hypothesis*: most objects die young, and those that survive tend to remain live for a long time [16]. To exploit this hypothesis, the heap is divided into at least two generations:

- The **young generation**, where newly allocated objects reside. Young generation collections are frequent but fast, since most young objects are short-lived.
- The **old generation**, where objects that have survived multiple young collections are promoted to. Old generation collections are less frequent but more expensive since live objects are more plentiful here.

ZGC adopted generational collection starting with JDK 21 [5]. In generational ZGC, the young and old generations can be collected independently and therefore concurrently. This is relevant to reference processing because references in different generations are handled during different collection cycles.

2.2 Java Reference Types

In addition to ordinary (strong) references, Java provides *reference objects* in the `java.lang.ref` package [6, 8] that allow applications to reference objects without preventing their garbage collection. These reference types are:

1. **SoftReference** The referent field is cleared at the discretion of the GC, typically when memory is low. Commonly used for memory-sensitive caches.
2. **WeakReference** The referent field is cleared as soon as it becomes weakly reachable (i.e., no strong or soft references exist to it). Widely used in caching frameworks, canonical mappings, and listener registrations.
3. **FinalReference** An internal JVM type (not part of the public API) used to implement the `finalize()` mechanism. Objects with finalisers are wrapped in a `FinalReference` so that the GC can schedule finalisation before reclaiming the object. This scheduling is done via a `ReferenceQueue`, which means that `FinalReferences` always have an associated `ReferenceQueue`.

4. **PhantomReference** The referent is never accessible through the reference; `get()` always returns `null`. Used for post-mortem cleanup actions, often to avoid the pitfalls of finalisation. Phantom references must be registered with a `ReferenceQueue` to be useful.

All four types extend the abstract class `Reference<T>` (in the `java.lang.ref` package), which defines the key fields of the reference object and the API for retrieving the referent, clearing the referent field, and checking if it is enqueued.

2.2.1 The Reference Object Lifecycle

Each `Reference` object maintains several fields that track its state [8]:

- **referent field** The referenced object. The GC may clear this field (set it to `null`) when the referent becomes eligible for collection according to the reference's type.
- **queue field** The `ReferenceQueue` this reference is registered with. If the reference was not created with a queue, this field points to the sentinel object `ReferenceQueue.NULL_QUEUE`.
- **next field** Used as a link pointer when the reference is enqueued in a `ReferenceQueue`.
- **discovered field** A field with dual purpose: during GC marking, it serves as a link in the garbage collector's internal *discovered list*; after processing, it is reused as a link in the VM's *pending list* that feeds the `ReferenceHandler` thread. Due to its first use, the GC can detect whether a reference has already been discovered by checking if this field is `null`.

A reference object transitions through the following states during its lifetime [17, 8]:

1. **Active** The reference has been created and the referent is still reachable (or not yet processed).
2. **Pending** The GC has determined the referent is eligible for clearing and has added the reference to the VM's pending list. The `ReferenceHandler` thread will process it.
3. **Enqueued** The `ReferenceHandler` thread has moved the reference into its associated `ReferenceQueue`, where the application can retrieve it via `poll()` or `remove()`.
4. **Inactive** The reference has been dequeued by the application, or was created without a queue and has been cleared. It is no longer tracked by the GC or queuing infrastructure.

An important observation is that references created *without* a `ReferenceQueue` can transition directly from Active to Inactive, bypassing the Pending and Enqueued states entirely. However, as discussed in Section 2.5, the default

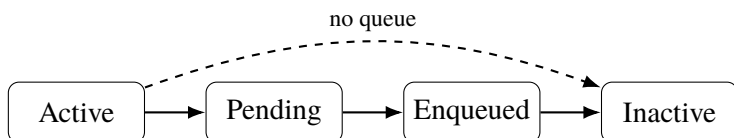


Figure 2.1. Lifecycle of Reference objects. References without a queue can bypass Pending and Enqueued.

implementation does not take advantage of this shortcut. The lifecycle and this direct shortcut path are illustrated in Figure 2.1.

2.2.2 WeakReference and ReferenceQueue

A `WeakReference` provides two constructors (with and without a `ReferenceQueue`), as shown in Listing 2.1 and implemented in OpenJDK [7].

```

1 // Without a queue - queue field set to NULL_QUEUE
2 public WeakReference(T referent) {
3     super(referent);
4 }
5
6 // With a queue
7 public WeakReference(T referent, ReferenceQueue<? super
8     T> q) {
9     super(referent, q);
10 }

```

Listing 2.1: *WeakReference* constructors

When a `WeakReference` is created with a `ReferenceQueue`, the application can be notified when the referent is collected. When created *without* a queue, the reference serves only as a conditional pointer, the application can call `get()` to check whether the referent is still alive, but receives no notification upon clearing.

The `ReferenceQueue` is implemented as a synchronized singly-linked list using the `next` field of the `Reference` objects. It provides `poll()` for non-blocking retrieval and `remove()` for blocking retrieval with an optional timeout [9]. This is the mechanism by which applications can react to the collection of referents, for example to clean up associated resources.

A key insight for this thesis is that Java explicitly supports queue-less weak references through the one-argument `WeakReference` constructor [7]. For workloads that only need liveness probing via `get()`, queue registration is unnecessary. In such cases, the VM still performs enqueue-path preparation work in the default pipeline, which motivated JDK-8029205 and the optimisation work in this thesis [12, 8, 11].

2.3 The JVM Execution Model

To understand how garbage-collector barriers are injected and why ZGC requires different components for each execution tier, it is helpful to examine HotSpot’s tiered execution model, which balances start-up latency against sustained throughput. Three execution tiers are relevant to this thesis:

1. **Interpreter.** At start-up, methods are first executed by the template interpreter, in which each bytecode is handled by generated assembly stubs. Interpretation provides the lowest throughput of the three tiers, but it has no Just In Time (JIT) compilation overhead and it gathers profiling data for later optimisation.
2. **C1 Compiler.** C1 is a fast JIT compiler that applies lightweight optimisations such as inlining, constant folding, and local simplifications. It compiles quickly, which makes it suitable for methods that are executed often enough to justify compilation but are not yet the hottest parts of the program.
3. **C2 Compiler.** C2 is the JVM’s aggressively optimising JIT compiler. It uses profiling data collected in earlier tiers to emit highly optimised native code for hot methods, applying transformations such as aggressive inlining, loop optimisations, and escape analysis.

In tiered compilation, methods move through these tiers as their execution frequency increases. A method typically starts in the interpreter, may then be compiled by C1, and can eventually be recompiled by C2 once sufficient profiling data has been collected.

Relevance to GC barriers.

Because a read barrier must intercept every load from the heap, and such loads occur in all three tiers, each tier requires its own barrier implementation. ZGC therefore ships three separate barrier components: `ZBarrierSetAssembler` generates barrier stubs embedded in the interpreter’s dispatch table, `ZBarrierSetC1` inserts barrier nodes during C1 intermediate representation construction, and `ZBarrierSetC2` integrates barriers into the C2 sea-of-nodes graph [15]. All three components obey the same decorator-driven barrier selection logic described in Section 2.4.3.

2.4 The Z Garbage Collector

ZGC is a concurrent, region-based, compacting garbage collector included in the JDK since JDK 11 (experimental) [3] and production-ready since JDK 15 [4]. Starting with JDK 21, ZGC supports generational collection, which is now the default mode [5]. The collector targets millisecond-scale pause times that remain bounded regardless of heap size, and supports heaps ranging from a few hundred megabytes to several terabytes [3]. Achieving these goals while performing the bulk of GC work concurrently is partly

realised by two closely related design decisions: encoding GC state directly in heap pointers, and using barriers to act on that state at every pointer access.

2.4.1 Coloured Pointers and Barriers

ZGC embeds metadata directly in the unused high bits of 64-bit heap pointers. These *colour bits* encode the state of a pointer relative to the current GC phase – for example, whether the pointed-to object has been remapped after relocation, or which generation it belongs to [3, 18]. Because the state is carried in the pointer itself, the GC can determine the status of a reference without loading the object’s header or consulting external tables.

To act on this encoded state, ZGC inserts barriers at every oop load and store. On a load, the barrier reads the colour bits of the retrieved pointer and checks whether they encode a *good* state for the current GC phase. If the pointer is stale – for example because the object has been relocated since the pointer was written – the barrier’s slow path applies the necessary fixup (remapping the pointer to the new address) before returning the address to the caller [3, 15]. Because these fixups happen lazily on access rather than eagerly during relocation, the collector can relocate objects concurrently without stalling the application.

Store barriers serve a complementary role: before overwriting a heap field, the barrier records the field’s previous value in a per-thread store-barrier buffer. This record is consumed by the marking phase to satisfy ZGC’s snapshot-at-the-beginning (SATB) invariant – ensuring that objects that were reachable at the start of a marking cycle are not missed – and by the remembered-set mechanism that tracks cross-generational pointers in the generational case [15].

2.4.2 Concurrent Phases and Parallel Workers

A ZGC collection cycle interleaves short stop-the-world pauses, used for root scanning and phase synchronisation, with longer concurrent phases in which marking, reference processing, and relocation proceed alongside running application threads [3, 11]. The load and store barriers described above are the mechanism that keeps the heap consistent during these concurrent phases: application threads that load a stale pointer automatically fix it up, and stores that would break the marking invariant are recorded for GC consumption.

Within each phase, ZGC distributes its work across multiple GC worker threads. To minimise contention, workers maintain their own thread-local data structures – such as per-worker discovered lists during reference discovery – and merge results into shared state only at well-defined synchronisation points [19, 11]. This design avoids the compare-and-swap contention that would arise from multiple threads updating a single shared resource.

2.4.3 Access Decorators

Throughout the HotSpot codebase, every load or store is annotated with a *decorator set*: a compile-time or runtime bitmask of type `DecoratorSet` (a 64-bit integer) whose set bits describe the semantics of the access [20]. The active `BarrierSet` implementation (e.g. `ZBarrierSet`) inspects this bitmask to decide which barrier variant to invoke, allowing a single generic `HeapAccess<decorators>::load()` call site to resolve to the correct low-level operation at both compile time (for template-specialised paths) and runtime (for dynamically dispatched paths).

The decorator flags are grouped by concern:

- **Reference strength** bits control how the barrier treats reachability. An access with `ON_WEAK_OOP_REF` tells ZGC that the field being read is the referent of a `WeakReference`: the barrier must return null if the referent has been collected, rather than reviving it. The default, when no strength decorator is specified, is `ON_STRONG_OOP_REF`.
- **Access location** bits indicates whether the access targets a heap object field or an off-heap native data structure. Some barrier actions are conditioned on `IN_HEAP`.
- **Barrier strength** bits control how strong barriers should be applied. `AS_RAW` bypasses GC barriers entirely and reads the raw memory value, which is necessary in certain internal GC paths. `AS_NO_KEEPLIVE` performs a correctly remapped access without keeping any objects alive, used in GC-internal scanning paths. `AS_NORMAL` (the default) invokes the full barrier.
- **Memory ordering** bits map to C++ memory-order semantics for the underlying atomic or plain load or store.

For ZGC, when `WeakReference.get()` is executed the referent field load carries `ON_WEAK_OOP_REF`. ZGC's barrier for this decorator checks the colour bits of the loaded pointer and if the pointer is already good (the referent is live and remapped or null), the address is returned directly. If not, the barrier's slow path is invoked. In the slow path, the liveness of the referent is checked and the barrier either returns a coloured null if the referent is no longer live, or a correctly coloured pointer if the referent is live [15, 20].

Store barriers are similarly decorator-driven. A normal store carries the decorator `ON_STRONG_OOP_REF` which triggers ZGC's store barrier, which records the old value in the store-barrier buffer (needed for the snapshot-at-the-beginning marking semantics and for maintaining remembered sets in the generational case).

2.5 Reference Processing Pipeline in ZGC

Reference processing is the mechanism by which the garbage collector handles `Reference` objects discovered during marking. The pipeline has three stages:

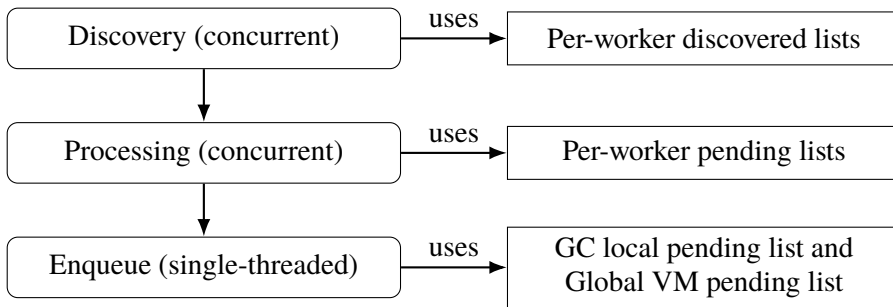


Figure 2.2. ZGC reference-processing pipeline and data flow from discovery to Java-level enqueue handling.

discovery, *processing*, and *enqueueing*. In ZGC, the first two stages run concurrently with the application during the marking phase, while the enqueue stage involves handing off references to the Java-level `ReferenceHandler` thread [11, 8]. An overview of this data flow is shown in Figure 2.2.

2.5.1 Discovery

During the concurrent marking phase, when ZGC encounters a `Reference` object (an instance of `SoftReference`, `WeakReference`, `FinalReference`, or `PhantomReference`), it evaluates whether the reference should be *discovered*, i.e. added to a list for later processing.

The discovery decision depends on several factors:

1. **Is the reference active?** Only active references are eligible. For non-`FinalReference` types, an inactive reference has a null referent field. For `FinalReference`, inactivity is indicated by a non-null next field.
2. **Is the referent still strongly reachable?** If the referent is already marked as strongly live, the reference does not need processing. The garbage collector checks this via its liveness information.
3. **Soft reference policy.** For `SoftReference` objects, the collector applies a policy (based on available memory and the reference’s last access time) to decide whether to keep the referent alive or clear the referent field [13, 21].
4. **Generational considerations.** In ZGC, only references residing in the old generation are discovered; young-generation references are always treated as strong [5, 11]. This is because reference processing introduces additional delay and uncertainty to the duration of the young-generation collection. By treating young references as strong, ZGC can complete young collections more predictably, which decreases the risk of the application running out of memory during a long young collection.

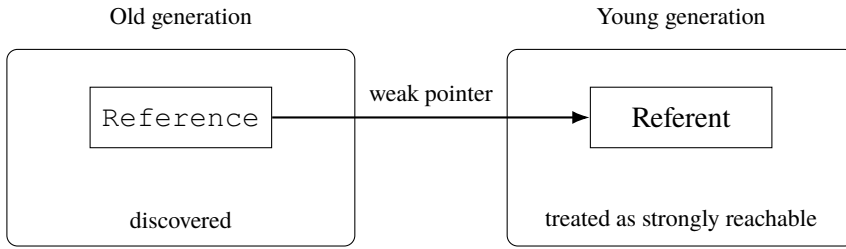


Figure 2.3. If an old-generation `Reference` points to a young referent, the young referent must remain alive during young collection.

Once a reference is selected for discovery, it is recorded on a *discovered list*. In ZGC, each GC worker thread maintains its own discovered-list head, so discovery is implemented as a cheap, non-atomic prepend to a per-worker list [11]. This avoids CAS contention found in shared processors where multiple threads update the same discovered head [13].

2.5.2 Processing

After marking completes, the discovered lists are processed. For each discovered reference, the collector determines the final fate of the referent:

1. If the referent field has become `null` (cleared by the application concurrently), the reference is dropped from the list.
2. If the referent is now strongly reachable (e.g. it was marked as alive after the reference’s discovery), the reference is dropped.
3. If the referent is still unreachable according to the reference’s strength, the referent field is **cleared** (set to `null`), and the reference is prepared for enqueueing.

In ZGC, clearing involves setting the `referent` field to a coloured null pointer via compare-and-swap (CAS) to ensure thread safety since application threads may be concurrently accessing the reference [15, 11].

After clearing, the reference is linked onto the thread-local pending list. This involves writing the reference’s address into the `discovered` field of the previously processed reference. Once the thread has processed all references in its discovered list, the thread-local list is atomically prepended to ZGC’s internal pending list [11].

Generational interactions are important during processing: when a `Refer-ence` object is old but its referent is young, the referent is treated as strongly reachable and the referent field must not be cleared. Processing must therefore verify generation membership and, if appropriate, ensure young-generation liveness is preserved (by marking the young referent), as illustrated in Figure 2.3 [11, 22].

2.5.3 Enqueue

After a reference's referent field has been cleared, the reference must be made available to the application if it was created with a `ReferenceQueue`. This happens through the *pending list* mechanism which feeds the `ReferenceHandler` thread [11, 8]. The handoff process is as follows:

1. The GC has built an internal pending list by linking cleared references together via their `discovered` field.
2. Under the heap lock, the GC atomically prepends its internal pending list onto the global VM pending list.
3. The `ReferenceHandler` daemon thread is notified.
4. The `ReferenceHandler` thread wakes up, atomically retrieves the pending list, and iterates over it. For each reference, it checks whether `queue != NULL_QUEUE`. If the reference has a real queue, it calls `queue.enqueue(this)`; otherwise it simply advances to the next reference.

This handoff reveals an inefficiency: references created without a `ReferenceQueue` are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be skipped [8, 11]. For workloads where the majority of weak references lack queues, this results in a considerable amount of extra work. This behaviour has been reported in an OpenJDK enhancement issue [12]. Avoiding enqueue-path work for references with `NULL_QUEUE` is one of the primary optimisation targets explored in this thesis.

3. Design and Implementation

Three optimisations were implemented to address the inefficiencies in ZGC's processing of `WeakReference` objects without a `ReferenceQueue`. These optimisations are:

- Skipping the enqueue path for weak references without a `ReferenceQueue`, allowing them to be processed without being added to the pending list for the `ReferenceHandler` thread.
- Replacing the intrusive linked list used for discovered references with a dynamic array for weak references without a `ReferenceQueue`.
- Optimising the codepath for clearing references, to eliminate unnecessary operations for weak references without a `ReferenceQueue`.

In addition, an exploratory weak-field implementation was developed to evaluate an alternative way of expressing weak semantics that avoids the overhead of `WeakReference` objects entirely.

3.1 Skipping Enqueue Path

As described in Section 2.5.2, ZGC enqueues discovered references to the internal pending list regardless of whether they have a `ReferenceQueue`. As discussed in Section 2.5.3, this is inefficient for weak references without a queue. The reference-processing pipeline was therefore changed so that weak references without a `ReferenceQueue` can bypass the enqueue path entirely. The expected effect is less work for GC threads during reference processing, and therefore lower reference-processing time and GC cycle time.

3.1.1 Implementation of Skipping Enqueue Path

In order to skip enqueueing weak references without a `ReferenceQueue` the `ZReferenceProcessor` class was modified to check if the queue field of a weak reference being discovered is equal to the sentinel value `ReferenceQueue.NULL_QUEUE` before adding it to the discovered list. If the queue field is equal to `ReferenceQueue.NULL_QUEUE` (the weak reference was not created with a queue), it is added to a separate discovered list for weak references without a `ReferenceQueue` instead of being added to the regular discovered list. This separate discovered list is then processed independently by the GC threads and none of its weak references are enqueued to the pending

```

1 bool discover_reference(oop ref_obj, ReferenceType type)
2 {
3     zaddress ref = to_zaddress(ref_obj);
4
5     if (!should_discover(ref, type)) {
6         return false;
7     }
8
9     if (type == REF_WEAK && !has_reference_queue(ref)) {
10        weak_no_queue_list_per_worker.append(ref);
11    } else {
12        discovered_list_per_worker.append(ref);
13    }
14
15    mark_discovered(ref);
16    return true;
17 }

```

Listing 3.1: *Pseudocode of the null-queue fast path during discovery. Queue-less weak references are routed to a separate discovered list and processed by a no-enqueue pipeline (see Appendix B.2).*

list for the `ReferenceHandler` thread. Each GC thread maintains its own separate discovered list for weak references without a `ReferenceQueue` to avoid contention between threads. See Listing 3.1 for a pseudocode summary and Appendix B.2 for the actual implementation.

The reason for separating the discovered lists instead of simply skipping enqueueing for weak references without a `ReferenceQueue` is that two unique reference processing functions can then be used for the two lists: one that includes enqueueing and one that does not. This allows for more efficient processing of the weak references without a `ReferenceQueue`, as it avoids unnecessary checks for whether the weak reference has a queue during processing.

3.2 Dynamic Array for Discovered References

During processing, ZGC iterates over the discovered references. As described in Section 2.5.1, the baseline design stores them in an intrusive linked list [11]. Traversing such a list has poor cache locality [14], because successive references may be scattered throughout the heap. For weak references without a `ReferenceQueue`, the linked list was therefore replaced with a dynamic array. The expected benefit is better spatial locality during scanning, which should reduce time spent waiting on memory loads during processing and thereby lower both reference-processing time and GC cycle time.

3.2.1 Implementation of Dynamic Array for Discovered References

The implemented dynamic array, called `ZWeakRefArray`, stores discovered references in a contiguous layout rather than linking them via the `discovered` field. The array is allocated on the C heap and grows dynamically as needed.

Each GC worker thread maintains its own `ZWeakRefArray` instance to avoid contention. During discovery, when a reference should be added to a discovery list, its data is appended to the worker's array instead of being appended to a linked list. The reference's `discovered` field is still set (to a self-referencing pointer) to mark the reference as discovered, preventing duplicate discovery.

During processing, the array is iterated sequentially using index-based access. Because the elements are stored contiguously in memory, this iteration improves cache line utilisation compared to following pointers scattered across the heap [14]. After processing, the array is cleared for the next GC cycle but retains enough capacity to at least hold the same number of references that were processed but not made inactive. This avoids unnecessary reallocations in subsequent cycles when the number of discovered references is relatively stable.

See Listing 3.2 for a pseudocode summary and Appendices B.1 and B.2 for the actual implementation.

3.3 Optimised Clear Path

The standard reference processing path in ZGC uses general-purpose functions designed to handle all reference types uniformly. This generality introduces per-reference overhead that is unnecessary when processing only weak references without a `ReferenceQueue`, where the reference type is always `REF_WEAK` and enqueueing is never required.

The standard function for determining whether a reference's referent should be cleared performs several operations per reference [11, 15]:

1. A load barrier to read the referent from the reference object.
2. A virtual call to determine the reference type (soft, weak, final, or phantom).
3. A compare-and-swap (CAS) operation via a ZGC barrier to atomically set the referent field to a coloured null pointer.

For weak references without a `ReferenceQueue`, each of these operations can be simplified or eliminated:

1. When combined with the dynamic array optimisation, the referent field address and its value can be pre-loaded during discovery and stored in the array entry. This avoids redundant memory loads during processing. Pre-loading is safe because the only concurrent application-level operations

```

1 void discover_reference(oop ref_obj, ReferenceType type)
2 {
3     //...
4     ZWeakRefData data;
5     data.set_from_ref(ref);
6     worker_array.append(data)
7     //...
8 }
9
10 void process_worker_discovered_weak_refs_without_queue (
11     ZWeakRefArray& arr) {
12     size_t dropped = 0;
13     for (size_t i = 0; i < arr.length(); i++) {
14         const ZWeakRefData& data = arr.at(i);
15         data.mark_not_discovered();
16
17         if (try_make_inactive(data)) {
18             cleared_weak_no_queue_count++;
19         } else {
20             dropped++;
21         }
22     }
23     arr.clear_and_reserve(dropped);
24 }

```

Listing 3.2: *Pseudocode for the ZWeakRefArray-backed queue-less weak-reference path. Discovery appends pre-fetched fields and processing scans the array sequentially (see Appendices B.1 and B.2).*

on a `Reference`'s referent are clearing and enqueueing, both of which set the referent field to null rather than to a new non-null value [8, 11].

2. The reference type is known to be `REF_WEAK` at the call site, so the virtual call to determine the type can be eliminated.
3. The only concurrent modification to the referent field is an application thread setting it to null [8, 15, 11]. Because no thread can set the field to a new non-null value, the CAS operation for clearing the referent can be replaced with a simple store of a coloured null pointer.

These simplifications reduce both instruction count and memory-access overhead for each weak reference without a `ReferenceQueue` during processing, which should in turn reduce reference-processing time and GC cycle time.

3.3.1 Implementation of Optimised Clear Path

The implementation introduces a new function for determining whether a queue-less weak reference's referent should be cleared (see Appendix B.2). This function takes a data struct (either from the dynamic array or constructed on the fly from the linked list) containing the pre-fetched field addresses and values. This new function has similar semantics to the standard function and the ZGC barriers it uses, but it is specialised for the queue-less weak reference case and can therefore use more efficient operations. See Section 2.4 for more details on ZGC barriers.

The new function checks whether the referent is strongly live via metadata in ZGC's heap management. If the referent is not strongly live and resides in the old generation, its referent field is cleared using a direct pointer store rather than a CAS. This is safe under the constrained update model described above. If the referent is still live, either because it is strongly marked or because it resides in the young generation, the referent field is recoloured with the correct pointer metadata bits. That recolouring still has to use CAS so that a null value is not overwritten with a non-null value. Additionally, if the referent is young and the young generation is currently in its mark phase, the referent is marked to preserve cross-generational liveness [11, 15, 22].

The `discovered` field is also cleared using a direct store to the field address rather than through an atomic access function, since no other thread modifies this field after discovery in this processing path [11].

See Listing 3.3 for a pseudocode summary and Appendix B.2 for the actual implementation.

3.4 Weak Fields

The three core optimisations above operate within the existing `WeakReference` object model. However, that model has an inherent structural overhead: each weak relation requires a separate Java object with queue and list-link metadata, and the GC must still discover and process that object. To answer Research Question 2, an exploratory weak-field mechanism was implemented to test whether expressing weak semantics directly on object fields can reduce this overhead. The mechanism allows developers to annotate object fields as weak, indicating that the GC should treat those fields with weak semantics. The GC then applies weak semantic rules directly to the annotated field without requiring a separate `WeakReference` object. This is expected to slightly reduce reference-processing overhead and to reduce Java heap usage by eliminating the need for separate `WeakReference` objects. The reduction in Java heap usage should also reduce GC cycle time, since garbage-collection cost is broadly proportional to the number of live objects [2].

3.4.1 Implementation of Weak Fields

The implementation includes changes in three layers of the JVM: a Java-level annotation, GC-side field discovery and processing, and compiler/interpreter support for the annotation metadata.

Annotation

A new runtime-retained field annotation, `@java.lang.ref.weak`, was added to the `java.lang.ref` package. The annotation targets only fields and is retained at runtime so that the VM can inspect it during class loading. See Listing 3.4 for the annotation definition. During class-file parsing, the VM recognises the `@weak` annotation and sets a `weak` flag in the field's metadata.

Discovery

Weak-field discovery occurs during ZGC's old-generation marking phase. As the marking closure iterates over each object's reference fields, it checks whether the current field carries the `weak` flag in its metadata. If the field is annotated as `weak`, the marking closure diverts it to the weak-field discovery path instead of applying the normal strong mark barrier [11].

The discovery function first checks whether the field should be discovered at all. Fields that already point to null or to a strongly live object (identified by their ZGC pointer metadata bits) are skipped, since no clearing action will be needed for them. Fields in young-generation objects are also skipped, because young-generation weak-field processing is not supported by this implementation. If the field passes these checks, its address and current value are recorded as a `ZWeakFieldData` struct and appended to the GC worker thread's per-worker `ZWeakFieldArray`. See Listing 3.5 for a pseudocode summary and Appendix B.3.3 for the actual implementation.

Each GC worker thread maintains its own `ZWeakFieldArray` instance to avoid contention. The array is structurally identical to the `ZWeakRefArray` used for queue-less weak references: it stores entries contiguously on the C heap and grows dynamically (see Appendix B.3.2).

Processing

During the reference-processing phase, each GC worker iterates through its `ZWeakFieldArray` and applies a clearing function to each entry. The function uses the data recorded during discovery. If the field still contains the discovered value and the referent is no longer strongly live, the routine clears the field by storing a coloured null pointer. If the field still contains the discovered value but the referent remains live, it recolours the pointer with the correct metadata bits. If the field value has changed since discovery, no action is required, since only a strongly reachable referent could have been stored in the field after discovery. Apart from this extra check for post-discovery field updates, the function follows the same overall strategy as the optimised

clear path described in Section 3.3. After processing, the array is cleared but retains enough capacity to hold the entries that were not cleared, following the same capacity-retention strategy as `ZWeakRefArray`. See Listing 3.6 for a pseudocode summary and Appendix B.3.3 for the actual implementation.

Compiler and Interpreter Support

The `@weak` annotation metadata is also made available to the Just-In-Time compilers and interpreter. This is necessary to ensure that any loads or stores to a weak field outside GC processing are handled correctly with respect to weak semantics. The implementation adds a decorator to loads and stores that target a field carrying the `weak` flag in its metadata. ZGC then inserts barriers that apply the required semantics to all accesses with this decorator. Without the load decorator, a load from a weak field could return a non-null reference which referent had already been cleared by the GC. The store decorator is needed so that the GC does not record the previous value of the field, unlike in the default strong-reference case used to preserve the snapshot-at-the-beginning marking semantics. See Section 2.4.3 for more details on decorators and Appendix B.3.5 for the implementations.


```

1 bool try_make_inactive_fast(const ZWeakRefData& data) {
2     // if used with the dynamic array
3     zaddress referent = data.referent_addr;
4
5     // if used with the linked list
6     zaddress referent = load_barrier(data.
7         referent_field_addr);
8
9     // Young referents are implicitly kept alive by
10    generational rules.
11    if (is_young(referent)) {
12        if (young_mark_is_active()) {
13            mark_young_root(referent);
14        }
15        return false; // keep as active
16    }
17
18    // Old referent: clear only if not strongly live.
19    if (!is_strongly_live(referent)) {
20        *data.referent_field_addr = to_zpointer(zaddress
21            ::null);
22        return true; // became inactive
23    }
24
25    // Still live: recolour pointer metadata, but do not
26    // overwrite null.
27    zpointer observed = data.referent_field_value;
28    zpointer recoloured = remap_with_good_metadata(
29        referent);
30    cas_referent_if_non_null(data.referent_field_addr,
31        observed, recoloured);
32    return false;
33 }

```

Listing 3.3: Pseudocode for the specialised queue-less weak-reference inactive check. The old-generation clear path uses direct stores, while recolouring uses CAS to avoid null-to-non-null races (see Appendix B.2).

```

1 package java.lang.ref;
2
3 @Retention(RetentionPolicy.RUNTIME)
4 @Target(ElementType.FIELD)
5 public @interface weak {}

```

Listing 3.4: The `@weak` annotation. Applied to a field, it instructs the GC to treat the field value with weak-reference semantics (see Appendix B.3.1).

```

1 // During marking: the closure checks each reference
  field
2 void do_oop(oop* p) {
3     if (is_weak_annotated_field(p)) {
4         if (discover_weak_field(p)) {
5             return; // discovered, skip strong marking
6         }
7     }
8     // ... normal strong marking ...
9 }
10
11 bool discover_weak_field(oop* field) {
12     encountered_weak_fields_count++;
13
14     if (is_young(field)) {
15         return false; // young generation not supported
16     }
17
18     if (is_null(field) || is_strongly_live(field)) {
19         return false; // nothing to clear
20     }
21
22     discovered_weak_fields_count++;
23     ZWeakFieldData data(field);
24     worker_weak_field_array.append(data);
25     return true;
26 }

```

Listing 3.5: Pseudocode for weak-field discovery during marking. Annotated fields are diverted from the strong mark barrier and recorded in a per-worker array (see Appendix B.3.3).

```

1 void process_worker_discovered_weak_fields(
2     ZWeakFieldArray& weak_fields) {
3     size_t dropped = 0;
4     for (size_t i = 0; i < weak_fields.length(); i++) {
5         const ZWeakFieldData& data = weak_fields.at(i);
6         if (try_make_inactive_fast(data)) {
7             cleared_weak_fields_count++;
8         } else {
9             dropped++;
10        }
11    }
12    weak_fields.clear_and_reserve(dropped);
13 }

```

Listing 3.6: *Pseudocode for weak-field processing. Each entry is passed to the clean barrier, which clears dead referents and recolours live ones (see Appendix B.3.3).*

4. Evaluation

The implemented optimisations described in Chapter 3 were evaluated through custom microbenchmarks designed to synthetically target the reference processing pipeline. The benchmarks were executed on a supercomputer cluster under controlled conditions, and the resulting GC timing statistics and memory usage metrics were collected to enable comparison between the baseline and optimised variants.

4.1 Optimisation Variants

Nine optimisation variants were evaluated in total. Eight of them cover all combinations of the three `WeakReference` pipeline optimisations described in Chapter 3: the optimised clear path, skipping enqueue for queue-less references via separate discovered lists, and the dynamic array. Table 4.1 shows which optimisations each variant enables. The `none` variant serves as the unmodified baseline. The remaining variants enable a subset of the three optimisations, allowing their individual and combined effects to be isolated. The `all` variant enables all three simultaneously. The ninth and final variant, `weak_fields`, corresponds to the `@weak` field annotation described in Section 3.4.

Each variant is implemented as a set of patches under `patches/<variant>/`. All variants except for `dyn_only` and `weak_fields` also include a shared base patch with infrastructure changes common to those optimisations. Table 4.2 lists the resulting diff sizes; full per-file breakdowns are available in Appendix C. A build script (`scripts/build_configs.sh`) automates applying the appropriate patches and building each JDK binary.

4.2 Performance Evaluation

To evaluate the performance impact of the optimisation approaches, four microbenchmarks were used under controlled conditions: two `WeakReference`-based benchmarks and two weak-field counterparts with analogous workload structure. The benchmarks and their execution environment are described below.

Table 4.1. *Optimisation variants and their enabled optimisations. A tick indicates the optimisation is active; a dash indicates it is disabled.*

Variant	Optimised Clear Path	Skip Enqueue	Dynamic Array
none	—	—	—
dyn_only	—	—	✓
sep_only	—	✓	—
sep_dyn	—	✓	✓
clear_path_only	✓	—	—
clear_path_dyn	✓	—	✓
clear_path_sep	✓	✓	—
all	✓	✓	✓
weak_fields	—	—	—

Table 4.2. *Files changed and diff size per variant relative to upstream. Variants marked with † include the shared base patch (8 files, +39/−17 lines) in their totals.*

Variant	Modified	New	Ins.	Del.	Net
all†	10	1	+436	−65	+371
clear_path_dyn†	10	1	+448	−87	+361
clear_path_only†	10	0	+233	−44	+189
clear_path_sep†	10	0	+256	−43	+213
dyn_only	2	1	+210	−43	+167
sep_dyn†	10	1	+398	−58	+340
sep_only†	10	0	+213	−42	+171
weak_fields	27	2	+940	−257	+683

4.2.1 Benchmark Design

Two benchmark families were used to evaluate the optimisation approaches. The first family isolates reference-processing cost around a single shared reference, and the second is a more realistic multi-object workload that introduces greater allocation and liveness variability. Each family was implemented both with `WeakReference` objects and with weak fields. In order to enable `WeakReference`-to-weak-field comparison, both benchmark families use holder objects that either contain a `WeakReference` or an annotated weak field.

Single-Object Benchmark

The single-object benchmarks allocate 10 million holder objects, each containing either a `WeakReference` object or an annotated weak field that points to a single shared target object. The holder objects are allocated and stored in a randomised order using a Fisher-Yates shuffle [23] to avoid artificial locality

Table 4.3. *Cluster node hardware specification*

Property	Value
Total nodes	80
CPU	AMD EPYC 9454P (Zen 4), 2.75 GHz
Physical cores	48
Threads	96
Memory	768 GiB
Scratch	1.7 TiB
GPUs	None

patterns. None of the weak references are created with a `ReferenceQueue`. After a brief holding period, the single strong reference to the target object is released and a GC cycle is triggered via `System.gc()`. Because all weak references become clearable simultaneously in one GC cycle, this benchmark maximises the per-cycle reference-processing load and isolates the cost of the processing pipeline itself.

Multi-Object Benchmark

The multi-object benchmarks represent a slightly more realistic workload. They allocate 1 million objects, each containing a randomly sized byte array payload (between 509 and 1097 bytes). Each object has a strong reference and a corresponding holder object with either a `WeakReference` object or an annotated weak field. The allocation and storing order of both strong references and holder objects are randomised using separate Fisher-Yates shuffles [23] to avoid artificial locality patterns.

Unlike the single-object benchmark, the strong references are not all released at once. Instead, the benchmark proceeds through five rounds of clearing. In each round, 20% of the strong references are nulled in a randomised order, making the corresponding objects eligible for garbage collection. After each round, `System.gc()` is called explicitly to trigger a collection cycle. This gradual release pattern means that weak references and weak fields are discovered and processed across multiple GC cycles rather than in a single burst.

4.2.2 Execution Environment

The benchmarks were executed on UPPMAX, a supercomputer cluster at Uppsala University. Table 4.3 lists the hardware specification of the cluster nodes. Each variant was run using its pre-built JDK binary (see Section 4.1). Table 4.4 lists the JVM options used in all benchmark runs.

A 10-second cooldown period was inserted between variant runs to allow the system to return to a stable state.

Table 4.4. *JVM options used in all benchmark runs*

Option	Purpose
-Xms100g -Xmx100g	Fixed 100 GB heap
-XX:+UseZGC	Enable ZGC
-XX:InitialTenuringThreshold=1	Promote objects to old generation after one cycle
-XX:MaxTenuringThreshold=1	Cap tenuring at 1
-XX:-ZProactive	Disable proactive GC
-XX:ZCollectionIntervalMajor=315360000	Effectively disable timer-driven major collections
-XX:+ZCollectionIntervalOnly	Suppress allocation-pressure-driven major collections
-Xlog:gc+stats,gc+ref	GC statistics and reference processing logs
-XX:NativeMemoryTracking=summary	Native Memory Tracking

4.2.3 Measurement Methodology

Each benchmark was run for a total of 100 iterations. Jobs were submitted via SLURM and allocated one node running in exclusive mode. Iterations were split across 4 parallel instances. Each instance was pinned via `taskset` to a set of 10 JVM cores and 2 auxiliary cores (12 cores per instance), preventing interference between concurrently running instances. Each instance ran its share of the 100 iterations preceded by 1 warm-up iteration whose results were discarded. The warm-up executed the full benchmark workload for every variant. Within each measurement iteration, all nine optimisation variants were run in sequence with a 10-second cooldown between each. This interleaved ordering mitigates systematic drift in system conditions affecting one configuration more than others.

During each run, a memory monitoring script samples memory metrics from the process at high frequency. See Table 4.5 for the memory metrics collected.

In addition to the continuous memory samples, GC statistics are parsed from the log files of each run. For each GC metric, the parser extracts the total average and total maximum values reported by ZGC per run. The single-object benchmark concentrates its entire weak-reference load into a single collection cycle, so the per-run *maximum* is the relevant statistic. The multi-object benchmark spreads work across several cycles, so the per-run *average* is used instead. Table 4.6 lists the GC log metrics collected. The rest of the GC log metrics were also collected but are not the focus of this evaluation; they are available on [INSERT LINK HERE](#).

Table 4.5. *Continuous monitoring metrics used in the measurement pipeline.*

Metric	Units	Description
RSS	kB	Process resident set size, representing total physical memory in use by the JVM process.
GC Committed Memory	kB	GC-related memory currently committed by the JVM.
Java Heap	kB	Java heap usage.

4.3 Presentation of Results

Results are presented for all nine variants listed in Table 4.1: the unmodified baseline (`none`), the seven `WeakReference` pipeline variants, and `weak_fields`. This enables two complementary comparisons:

- **Intra-model comparison** among the `WeakReference` variants, isolating the effect of each optimisation and their interactions relative to the baseline.
- **Inter-model comparison** between the best-performing `WeakReference` variants and the weak-field representation, evaluating whether avoiding `WeakReference` objects altogether offers additional benefit.

Each GC timing figure shows a composite plot: a violin plot (top) with the full per-run distribution and a median percentage-difference histogram (bottom) showing all variants relative to the `none` baseline, including `None` itself at 0%. The single-object benchmark uses per-run maxima; the multi-object benchmark uses per-run averages, following the rationale in Section 4.2.3. The median value is compared due to the non-symmetric distributions that in general skew the average higher. GC timing results are shown for five metrics for each benchmark: Major Collection (Figures 4.1 and 4.2), Old Generation (Figures 4.3 and 4.4), Young Generation – Promote All (Figures 4.5 and 4.6), Old Phase: Concurrent Mark (Figures 4.7 and 4.8), and Old Phase: Concurrent Process Non-Strong (Figures 4.9 and 4.10). Memory results are shown as median percentage-difference histograms of per-run maxima for three metrics: auxiliary GC memory, Java heap, and total process RSS, for both benchmarks (Figures 4.11 and 4.12). Remaining GC metrics are shown in Appendix E.

Table 4.6. *GC log metrics used in the measurement pipeline. For the single-object benchmark, per-run maximum values are used; for the multi-object benchmark, per-run averages are used.*

Metric	Units	Description
Major Collection	ms	End-to-end duration of major collections.
Old Generation	ms	Total time spent in old-generation collection work.
Old Phase: Concurrent Mark	ms	Time spent in old-generation concurrent marking.
Old Phase: Concurrent Process Non-Strong	ms	Time spent processing non-strong references in the old generation.
Old Phase: Concurrent Relocate	ms	Time spent in old-generation concurrent relocation.
Young Generation [†]	ms	Total time in young-generation collection. For multi-object, zero values (cycles without young-gen work) are excluded.
Young Phase: Concurrent Mark	ms	Time spent in young-generation concurrent marking.
Young Phase: Concurrent Relocate	ms	Time spent in young-generation concurrent relocation.

[†]Both benchmarks use “Young Generation: Young Generation (Promote All)”.

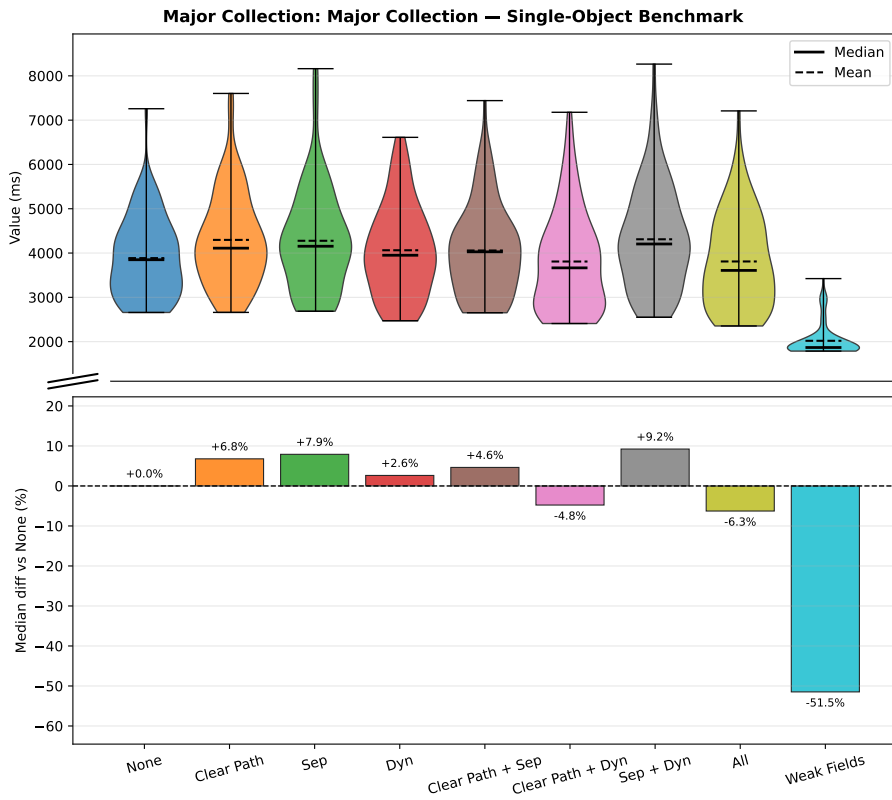


Figure 4.1. Major Collection timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

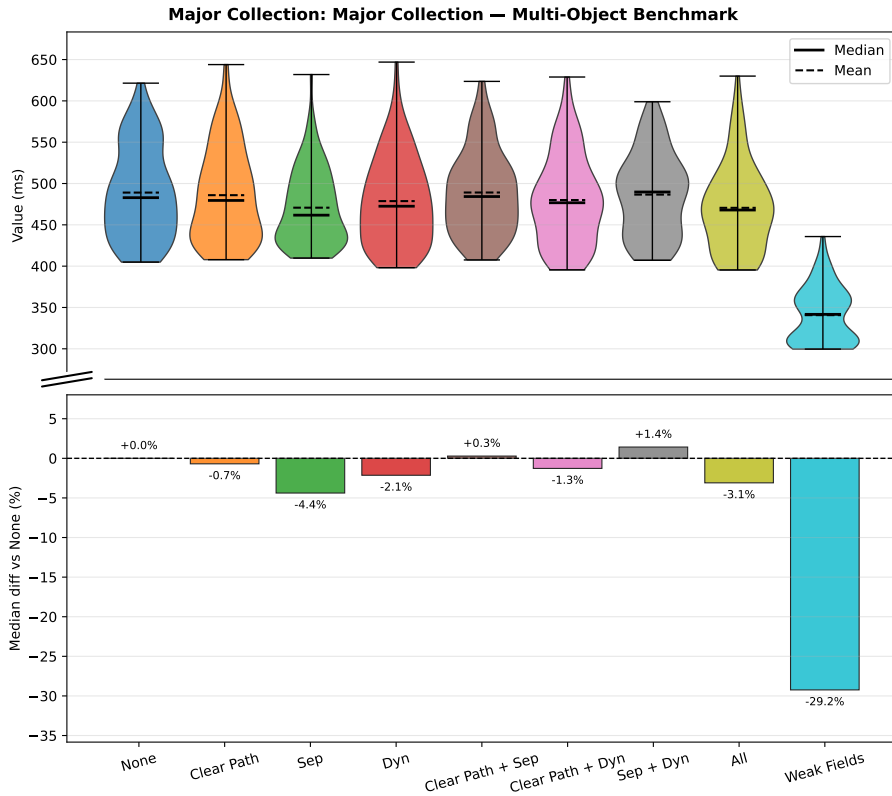


Figure 4.2. Major Collection timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

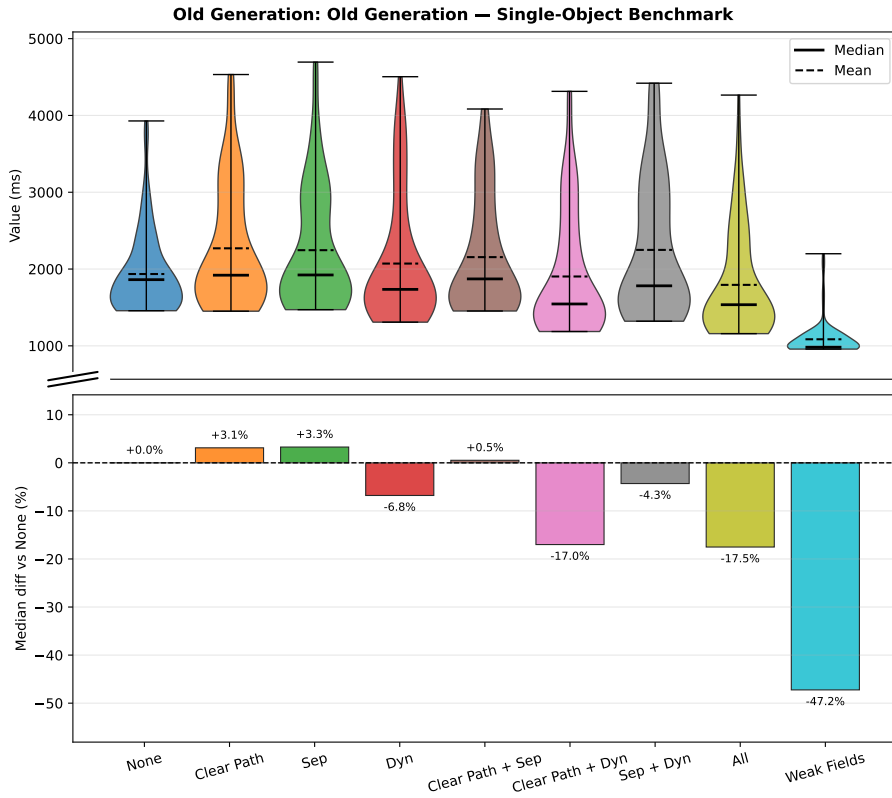


Figure 4.3. Old Generation timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

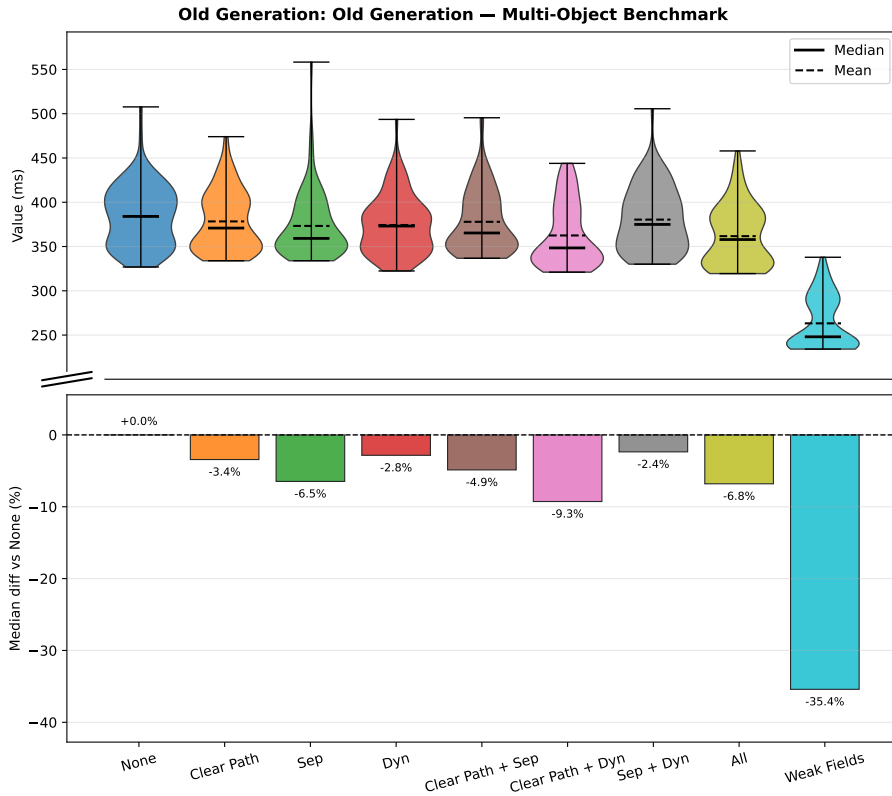


Figure 4.4. Old Generation timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

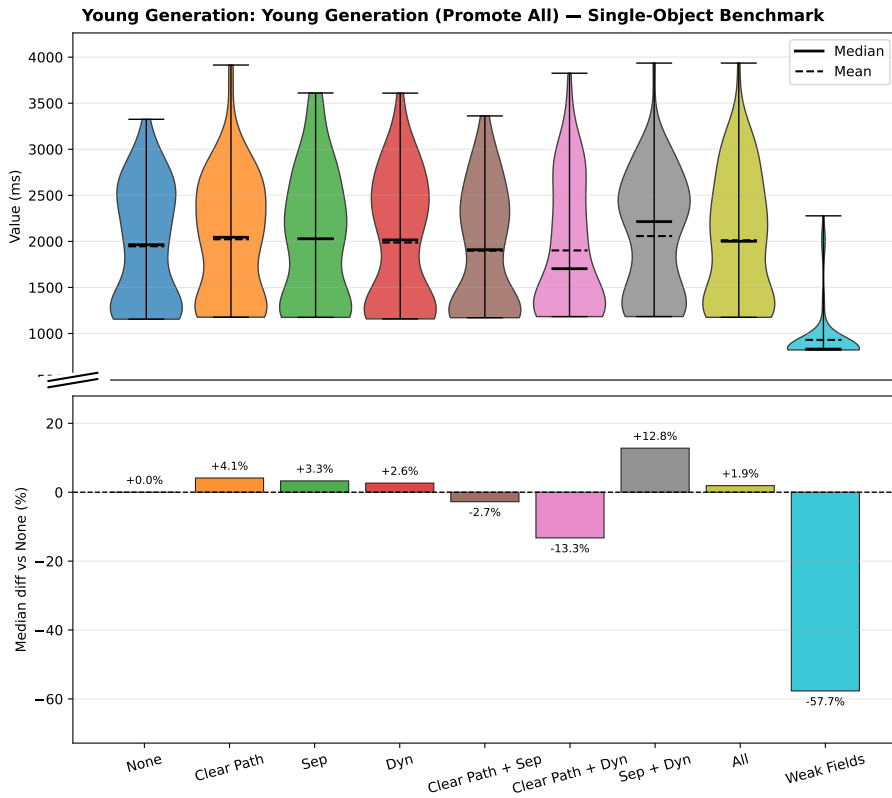


Figure 4.5. Young Generation (Promote All) timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

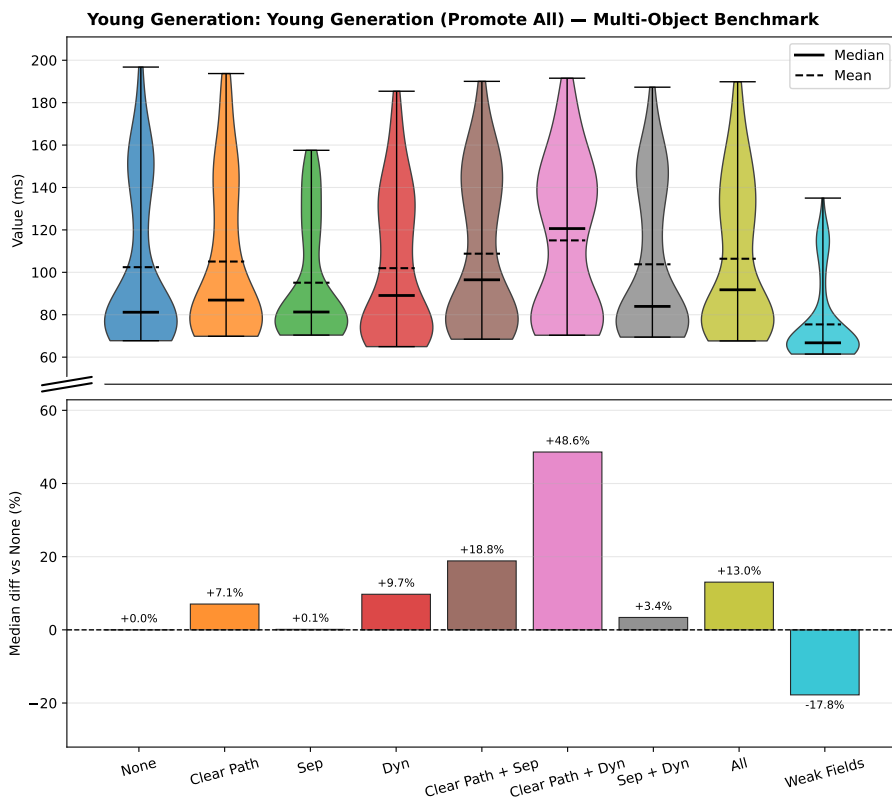


Figure 4.6. Young Generation (Promote All) timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

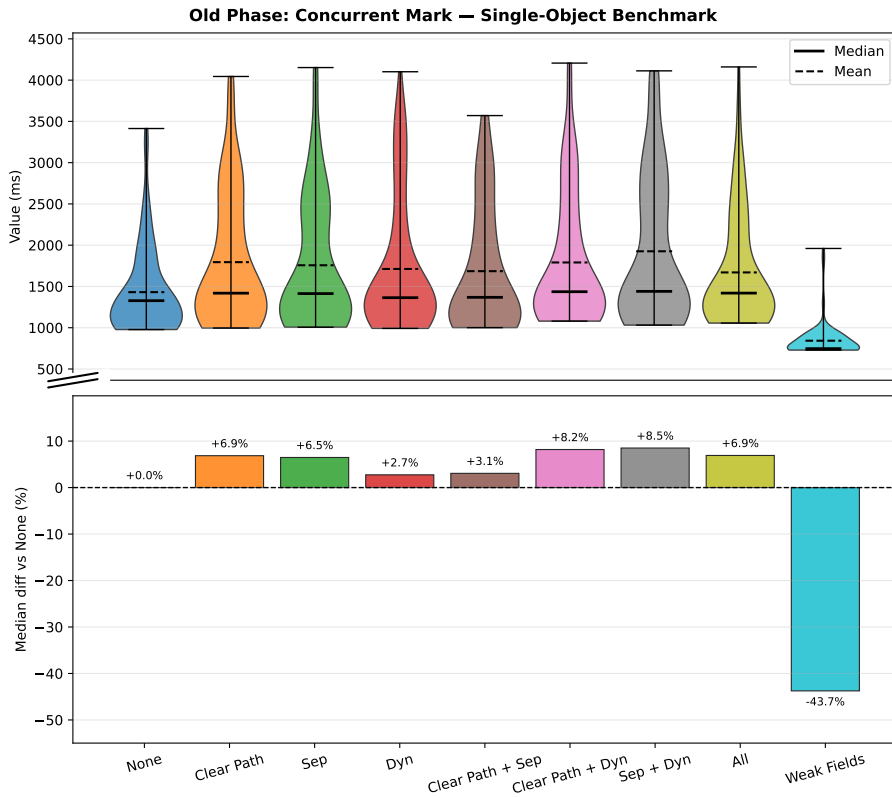


Figure 4.7. Old Phase: Concurrent Mark timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

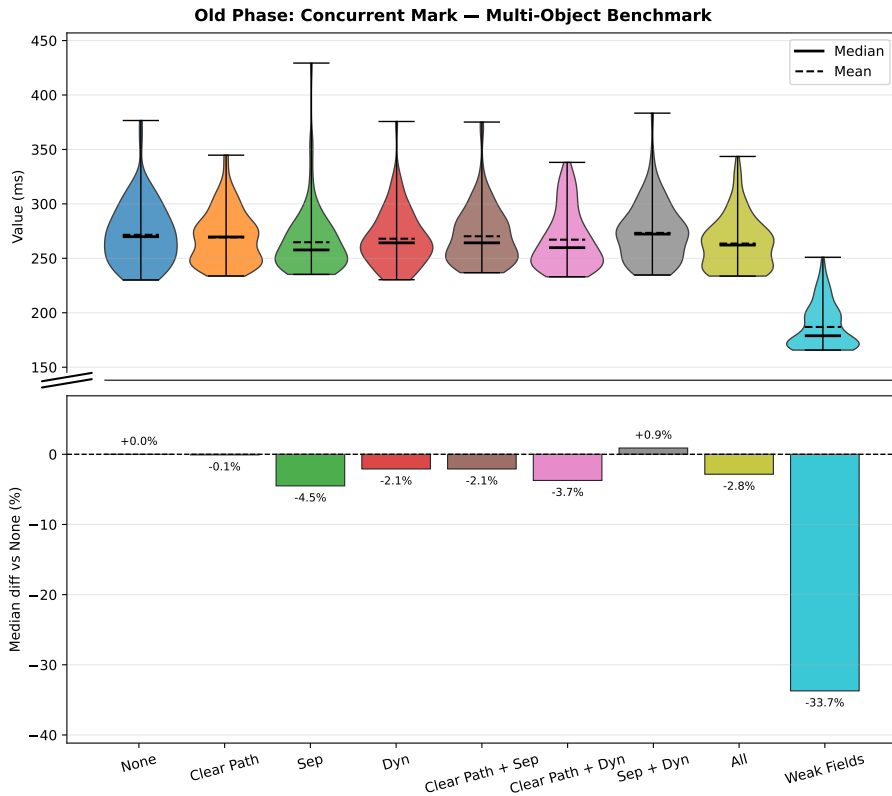


Figure 4.8. Old Phase: Concurrent Mark timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

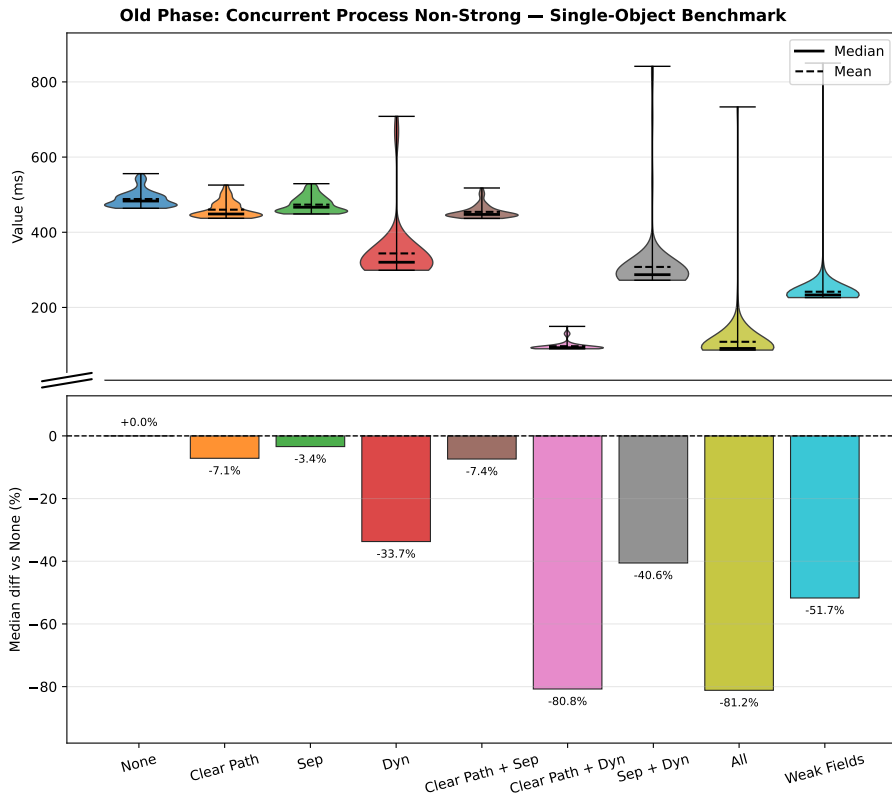


Figure 4.9. Old Phase: Concurrent Process Non-Strong timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

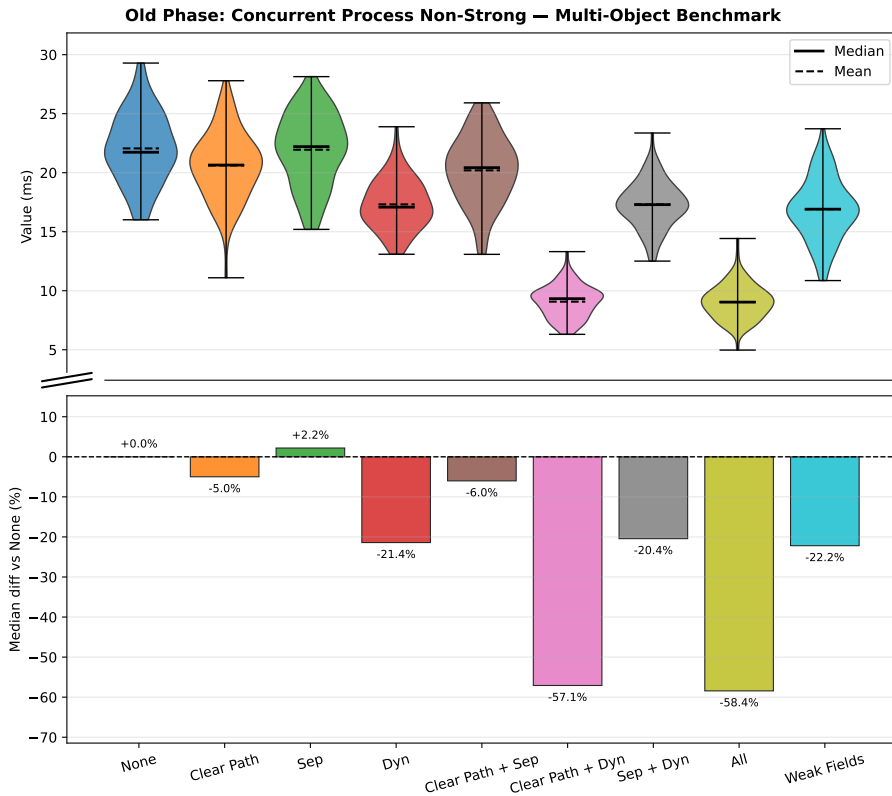


Figure 4.10. Old Phase: Concurrent Process Non-Strong timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

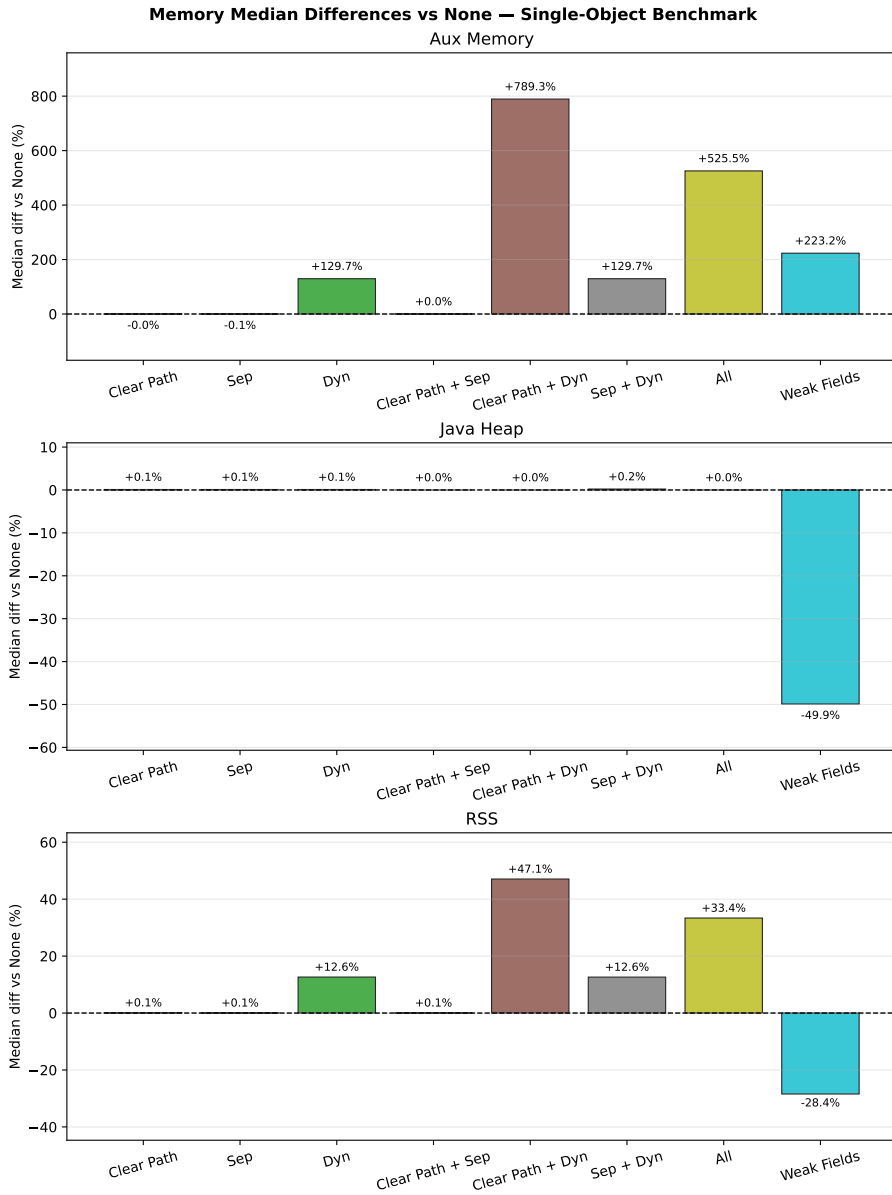


Figure 4.11. Median percentage difference in per-run maximum memory usage relative to the `none` baseline, single-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.

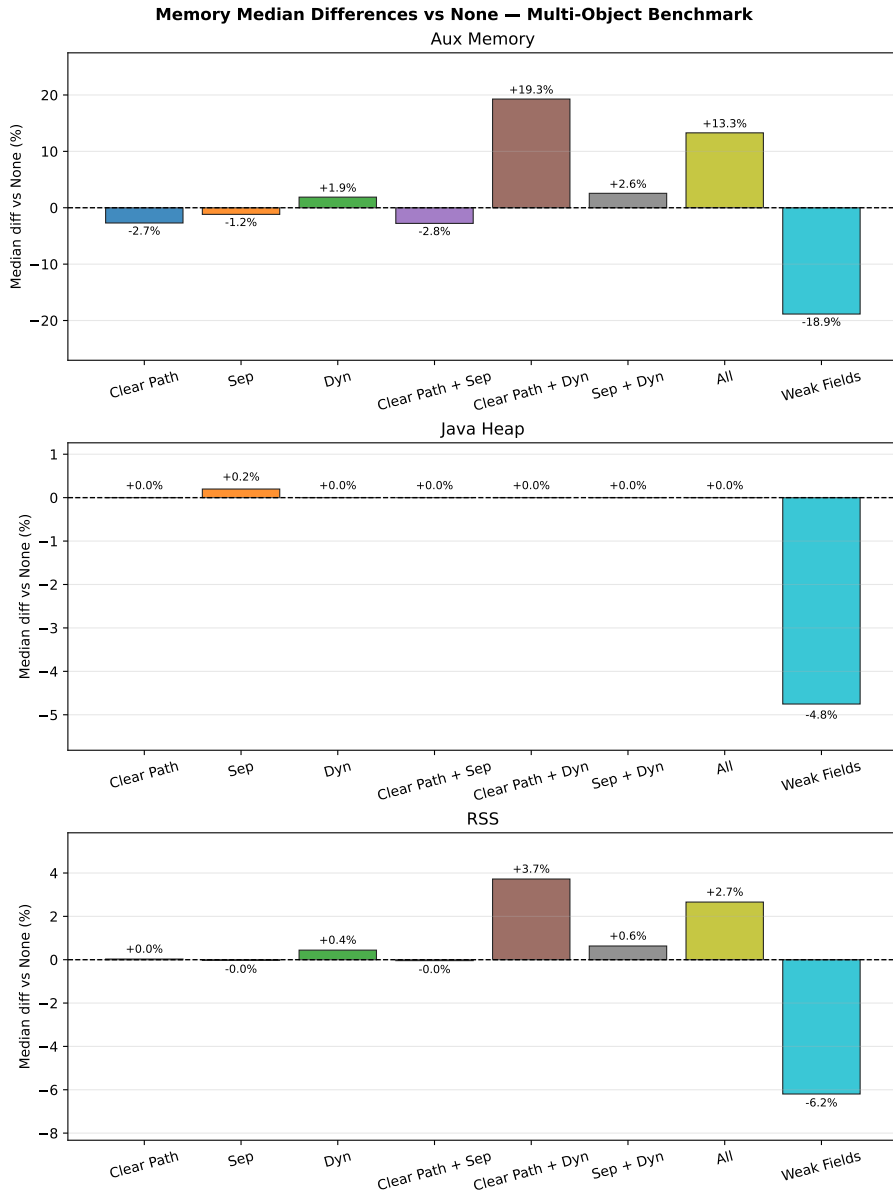


Figure 4.12. Median percentage difference in per-run maximum memory usage relative to the `none` baseline, multi-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.

5. Analysis of Results

The measurements presented in Section 4.3 show clear differences between configurations on some metrics and largely overlapping distributions on others.

5.1 Non-Strong Reference Processing

The metric most directly targeted by the implemented optimisations is `Concurrent Process Non-Strong`, which captures the total wall-clock time spent in ZGC’s concurrent reference-processing phase per GC cycle. Because this phase is the intended site of all three optimisations, the clearest differences between variants are observed here, making it the natural starting point for the analysis.

Figures 4.9 and 4.10 show the distributions of this metric for the single-object and multi-object benchmarks respectively. Full numerical summaries are given in Table 8.9.

5.1.1 Effect of the Clear Path and Dynamic Array

The most striking reduction is achieved by the `clear_path_dyn` variant. In the single-object benchmark its median non-strong processing time is 93 ms, compared to 483 ms for the unmodified `none` baseline, a reduction of approximately 81%. The `all` variant, which adds the skip-enqueue separation on top of `clear_path_dyn`, reaches a nearly identical median of 91 ms (a reduction of 81%). In the multi-object benchmark the pattern is consistent: `clear_path_dyn` achieves a median of 9.3 ms versus 21.7 ms for `none` (57% reduction), and `all` reaches 9.0 ms (58% reduction).

The near-equivalence of `clear_path_dyn` and `all` is a notable finding. Adding the skip-enqueue separation to an already active clear-path configuration (`sep` alone reduces time by only 3% in the single-object benchmark) adds negligible further benefit when the dynamic array and optimised clear path are both present.

The two individual components tell a different story. The `clear_path_only` variant, which enables only the optimised clear path, reduces the single-object median by a modest 7% (to 449 ms), while `dyn_only`, which enables only the dynamic array, achieves a 34% reduction (to 320 ms). When combined in `clear_path_dyn`, the effect far exceeds the sum of the individual contributions.

5.1.2 Effect of the Skip-Enqueue Separation Alone

The `sep_only` variant, which introduces the separate discovered list for queue-less weak references but does not include the dynamic array or the optimised clear path, produces negligible improvement on non-strong processing time. In the single-object benchmark its median is 467 ms, only 3% below the `none` baseline. In the multi-object benchmark the median is 22.2 ms, marginally higher than `none`'s 21.7 ms, which lies within measurement variability.

5.1.3 Comparison with Weak Fields

The `weak_fields` variant achieves a median non-strong processing time of 233 ms in the single-object benchmark and 16.9 ms in the multi-object benchmark. These figures correspond to reductions of 52% and 22% respectively relative to `none`, positioning `weak_fields` between the poorly performing `sep_only` and the leading `clear_path_dyn` configurations. Notably, `weak_fields` performs distinctly worse than `clear_path_dyn` and `all` on this metric despite showing large gains elsewhere (see Section 5.3).

5.2 Overall GC Collection Times

In contrast to the non-strong processing metric, the overall collection time metrics — major collection time and old-generation collection time — show broad, heavily overlapping distributions across all `WeakReference` variants, making it difficult to draw firm conclusions from pairwise comparisons within this group. Figures 4.1 and 4.2 illustrate this for the major collection metric. Numerical summaries for all variants are given in Tables 8.10 to 8.12.

5.2.1 WeakReference Variants

For the single-object benchmark, the median major collection time of `none` is approximately 3 849 ms with an interquartile range (IQR) of about 1 309 ms. The `all` variant has a median of 3 608 ms (6% below `none`) and the `clear_path_dyn` variant 3 666 ms (5% below `none`), but their IQRs of approximately 1 700 ms and 1 612 ms respectively overlap substantially with every other variant. Several variants with no reference-processing optimisations (`clear_path_sep`, `sep_only`, `clear_path_only`) actually show higher medians than `none`. In the multi-object benchmark the effect is even smaller in absolute terms, with medians ranging from 462 ms (`sep_only`) to 490 ms (`sep_dyn`) against a baseline of 483 ms; all distributions share essentially the same IQR of approximately 70–90 ms, making any ranking unreliable.

Among the `WeakReference` variants, `all` and `clear_path_dyn` tend to occupy the lower part of the collection-time distribution in both benchmarks, which is directionally consistent with their superior non-strong processing times. However, given the distribution overlap, this observation should be treated as indicative rather than conclusive.

5.2.2 Violin Plot Shape and the Bimodal Pattern

Many violin plots for the overall collection metrics exhibit a distinctly bimodal shape, with two separated density peaks rather than a single bell-shaped distribution. This pattern is visible in both benchmarks but is more pronounced for the single-object case, where per-run maxima are used. The distribution separation is not caused by any particular variant: all nine configurations exhibit it.

5.2.3 Single versus Multi-Object Benchmark

The single-object and multi-object benchmarks yield broadly consistent rankings across all metrics, giving confidence that the observed trends are not artefacts of one particular workload. The principal difference is in variability: the single-object benchmark produces substantially taller violin plots and a higher incidence of outliers. The relative ordering of variant medians is nonetheless consistent between the two benchmarks for all key metrics.

5.3 Inter-Model Comparison: Weak Fields versus WeakReference

The `weak_fields` variant stands apart from all `WeakReference` variants on the overall GC collection metrics (see Tables 8.10 to 8.12). In the single-object benchmark, its median major collection time is 1 868 ms, compared to 3 608–4 204 ms for all `WeakReference` variants, a reduction of approximately 49–54% relative to the weakest and strongest `WeakReference` performers respectively. For old-generation collection time the pattern is similar: 982 ms versus 1 536–1 923 ms (a reduction of 36–49%). In the multi-object benchmark the gap is smaller but still clear: 342 ms versus 358–490 ms for major collection (a reduction of 30–43%), and 248 ms versus 348–384 ms for old generation (a reduction of 35–55%). The `weak_fields` distributions are also notably tighter, with IQRs of approximately 195 ms (single, major) and 56 ms (multi, major) compared to 1 300–1 800 ms and 70–90 ms respectively for the `WeakReference` variants. The concurrent mark-follow phase shows a similar pattern: a median of 747 ms for `weak_fields` versus 1 328 ms for none in the single-object benchmark, a reduction of 44%.

5.4 Memory Behaviour

5.4.1 Auxiliary Memory Usage and the Dynamic Array

Figures 4.11 and 4.12 show the median percentage difference in per-run maximum GC-committed (auxiliary) memory relative to the `none` baseline across all configurations. Absolute values are listed in Table 8.13.

All configurations that include the dynamic array (`dyn_only`, `sep_dyn`, `clear_path_dyn`, and `all`) incur substantially higher auxiliary memory than variants that retain the intrusive linked list. In the single-object benchmark, the median per-run maximum for `dyn_only` and `sep_dyn` is approximately 167 MB of GC-committed memory, compared to 73 MB for the linked-list variants. The `clear_path_dyn` variant is the highest of all at 647 MB, and `all` reaches 455 MB. In the multi-object benchmark the absolute differences are smaller, with medians of 131–192 MB across all variants.

The `weak_fields` variant occupies an intermediate position. Its median auxiliary memory in the single-object benchmark is 235 MB, higher than the non-array `WeakReference` variants (73 MB) but lower than `clear_path_dyn`. In the multi-object benchmark it is the lowest of all configurations at 131 MB.

5.4.2 Java Heap Usage

The Java heap usage panel in Figures 4.11 and 4.12 shows the median percentage difference in per-run maximum heap usage relative to the `none` baseline. Absolute values are listed in Table 8.14.

The heap usage figures are essentially identical across all `WeakReference` variants (approximately 918–920 MB per-run median in the single-object benchmark and 1 010–1 014 MB in the multi-object benchmark).

The `weak_fields` variant, however, shows markedly lower heap usage: a median of approximately 460 MB in the single-object benchmark (about half of the `WeakReference` variants) and 962 MB in the multi-object benchmark.

6. Related Work

This chapter situates the thesis in prior work on weak semantics in managed runtimes by focusing on Go and Python. The purpose is to establish how these two language ecosystems expose weak references, how they combine weak semantics with lifecycle callbacks, and which trade-offs they document for runtime behaviour.

6.1 Go

Go exposes weak semantics through the standard-library package `weak`, which provides weak pointers that do not by themselves keep referents alive [24]. This design enables non-owning associations in cache-like or metadata-oriented structures without introducing strong retention edges.

For lifecycle callbacks, Go documents finalisation support through runtime facilities such as `SetFinalizer` [25]. Importantly, Go’s documentation emphasises that finalisers are not prompt resource-management mechanisms and should not be used as the primary correctness path for releasing resources. This guidance is relevant to weak semantics because weak references and finalisation are often combined in long-lived systems where cleanup latency and execution ordering can affect observable behaviour.

In summary, Go presents weak semantics as an explicit non-owning reference mechanism and presents finalisation as an auxiliary facility with carefully constrained expectations. Together, these choices prioritise clear ownership boundaries and conservative assumptions about callback timing.

6.2 Python

Python’s weak-semantics design, standardised in PEP 205, emphasises two use cases: cache construction and reduction of lifecycle coupling in graph-like object structures [26]. The language provides explicit weak-reference objects, weak mappings, and callback hooks on invalidation. A key contribution of the PEP is its explicit treatment of invalidation strategy and the cost implications of global weak-reference management.

At the API level, Python exposes weak references through the `weakref` module and supports optional callbacks when referents become unreachable [27]. This model is widely used for cache-like structures and registry patterns where identity tracking is needed without extending object lifetime.

Python’s documentation similarly cautions that finalisation-style mechanisms should not be treated as deterministic cleanup points [27]. This caution aligns with the broader runtime principle that weak semantics can express ownership intent, but cleanup timing remains GC-dependent.

6.3 Synthesis

Taken together, Go and Python frame weak semantics as an explicit programming abstraction rather than an implicit optimisation. Both ecosystems separate non-owning reachability from deterministic resource cleanup, and both make callback timing explicitly non-deterministic. For this thesis, these patterns are relevant because they underscore a central engineering constraint: weak-reference mechanisms should preserve clear semantic contracts while minimising avoidable runtime overhead in the common case.

7. Discussion

7.1 Interpretation of Results

The combination of the optimised clear path and dynamic array achieves a far greater reduction in non-strong processing time than either optimisation individually. The optimised clear path eliminates the enqueue step for queue-less weak references, routing them through a lighter processing path; once that lighter path is active, the dynamic array makes traversal of the resulting list faster through improved cache locality [14]. Each optimisation therefore amplifies the benefit of the other, producing a super-additive interaction. The near-equivalence of `clear_path_dyn` and `all` follows from this: when the clear path and dynamic array are already active, adding the separate-list mechanism provides little further gain because most of the processing work has already been removed. The `sep_only` variant confirms this from the other direction: routing references to a separate list without changing either the traversal structure or the per-reference work yields no meaningful speedup.

The `weak_fields` variant achieves a lower non-strong processing time than the baseline but falls short of `clear_path_dyn` because it bypasses the `WeakReference` pipeline entirely. The residual non-strong processing time for `weak_fields` reflects only reference types that remain in the pipeline, such as phantom and final references, rather than any optimisation of weak-reference traversal.

Despite the substantial reductions in non-strong processing time, the `WeakReference` optimisations have a limited measurable effect on total collection time. Reference processing accounts for only a fraction of the total GC cycle; the remainder is dominated by concurrent marking, relocation, and other phases that the optimisations do not touch. The overlapping distributions for major collection and old-generation time across `WeakReference` variants reflect this: once reference processing is no longer the dominant cost, further gains in that subphase translate to at most a small fraction of total cycle time.

The bimodal shape of many violin plots for overall collection metrics suggests that the benchmark encounters two distinct operating regimes. A plausible explanation is that some GC cycles carry a heavier concurrent mark-follow workload than others, perhaps due to object lifetimes or allocation patterns interacting with the timing of the explicit `System.gc()` call. This is consistent with similar bimodality observed in the mark-follow metric. The higher variability of the single-object benchmark relative to the multi-object

benchmark follows from its design: concentrating all 10 million references into a single GC cycle amplifies sensitivity to transient system effects such as cache state, NUMA placement, and thread scheduling. The multi-object benchmark spreads the reference load across five cycles, averaging out per-cycle variation.

The large advantage of `weak_fields` on overall collection metrics stems from eliminating `WeakReference` wrapper objects entirely. Each `WeakReference` object occupies heap space for its header, referent, queue, and discovered-list pointer fields, and with 10 million such objects in the single-object benchmark this amounts to approximately 460 MB of additional live heap. Eliminating these objects reduces GC pressure across all phases: fewer objects to mark, fewer pages to scan during relocation, and less metadata to maintain. The optimised clear path and dynamic array reduce per-reference cost within the processing pipeline, but they cannot offset the broader cost of maintaining 10 million reference objects throughout the GC cycle.

The elevated auxiliary memory of `clear_path_dyn` (647 MB median in the single-object benchmark) compared to `all` (455 MB) reflects an interaction between the optimisations. Without the separate-list mechanism, all queue-less references share a single dynamic array that retains its full allocated capacity throughout the GC cycle. With the separate-list mechanism active, references are split across two independently managed lists, bounding the peak size of each more tightly. In all cases, the additional GC-committed memory is negligible relative to the 100 GB fixed heap. The identical heap footprints of all `WeakReference` variants confirm that the optimisations do not affect the number of wrapper objects allocated; only `weak_fields`, which eliminates the wrapper objects themselves, reduces Java heap usage.

7.2 Future Work

Can the optimisations developed for ZGC be applied to other garbage collectors in the JVM, such as the G1 garbage collector, and if so, what performance improvements can be observed?

8. Conclusion

References

- [1] Oracle, “Available collectors.” <https://docs.oracle.com/en/java/javase/25/gctuning/available-collectors.html>, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-02-24.
- [2] Oracle, “Garbage collector implementation.” <https://docs.oracle.com/en/java/javase/25/gctuning/garbage-collector-implementation.html>, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-04-07.
- [3] OpenJDK, “Jep 333: Zgc: A scalable low-latency garbage collector (experimental).” <https://openjdk.org/jeps/333>, 2018. Accessed: 2026-03-12.
- [4] OpenJDK, “Jep 377: Zgc: A scalable low-latency garbage collector (production).” <https://openjdk.org/jeps/377>, 2020. Accessed: 2026-03-12.
- [5] OpenJDK, “Jep 439: Generational zgc.” <https://openjdk.org/jeps/439>, 2023. Accessed: 2026-03-12.
- [6] Oracle, “java.lang.ref (java se 25).” <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html>, 2025. Accessed: 2026-03-12.
- [7] OpenJDK, “Openjdk source: java.lang.ref.weakreference.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/WeakReference.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [8] OpenJDK, “Openjdk source: java.lang.ref.reference.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/Reference.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [9] OpenJDK, “Openjdk source: java.lang.ref.referencequeue.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/ReferenceQueue.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [10] OpenJDK, “Openjdk source: java.util.weakhashmap.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/util/WeakHashMap.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [11] OpenJDK, “Openjdk source: Zgc reference processor (zreferenceprocessor.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zReferenceProcessor.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [12] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.” <https://bugs.openjdk.org/browse/JDK-8029205>, n.d. Accessed: 2026-02-24.
- [13] OpenJDK, “Openjdk source: Shared reference processor (referenceprocessor.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/shared/referenceProcessor.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.

- [14] U. Drepper, “What every programmer should know about memory,” tech. rep., Red Hat, Inc., 2007. Accessed: 2026-04-07.
- [15] OpenJDK, “Openjdk source: Zgc barrier implementation (zbarrier.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zBarrier.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [16] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981. Accessed: 2026-02-24.
- [17] Oracle, “Reference (java se 25).” <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/Reference.html>, 2025. Accessed: 2026-03-12.
- [18] OpenJDK, “Openjdk source: Zgc address representation (zaddress.hpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zAddress.hpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [19] OpenJDK, “Openjdk source: Zgc worker infrastructure (zworkers.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zWorkers.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [20] OpenJDK, “Openjdk source: Access decorators (accessdecorators.hpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/oops/accessDecorators.hpp>, 2026. GitHub code blob. Accessed: 2026-04-16.
- [21] OpenJDK, “Openjdk source: Reference policy (referencepolicy.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/shared/referencePolicy.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [22] OpenJDK, “Openjdk source: Generational zgc implementation (zgeneration.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zGeneration.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [23] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd ed., 1997. Section 3.4.2, Algorithm P.
- [24] The Go Authors, “weak package – go standard library documentation.” <https://pkg.go.dev/weak>, 2026. Accessed: 2026-04-22.
- [25] The Go Authors, “runtime.setfinalizer – go standard library documentation.” <https://pkg.go.dev/runtime#SetFinalizer>, 2026. Accessed: 2026-04-22.
- [26] F. L. Drake, “Pep 205 – weak references.” <https://peps.python.org/pep-0205/>, 2000. Python Enhancement Proposal. Accessed: 2026-04-22.
- [27] Python Software Foundation, “weakref – weak references (python 3 documentation).” <https://docs.python.org/3/library/weakref.html>, 2026. Accessed: 2026-04-22.

Appendix A.

Usage of Generative Artificial Intelligence Tools

The generative artificial intelligence (GenAI) tool GitHub Copilot was used as support during this thesis. It was used under a student licence via Uppsala University. The tool was used to accelerate practical work and improve writing quality, while all final technical decisions, implementation choices, and evaluations remained under the author's control.

A.1 Use During Implementation

GenAI was used as support during coding of the implementation, primarily for discussing alternative implementation approaches, suggesting code edits through auto-complete, and implementing refactorings of already functioning code.

A.2 Use for Tests and Scripts

GenAI was used to completely generate several test files and automation scripts used in this project, including scripts for benchmarking workflows and result-processing utilities. These generated artefacts were reviewed and validated through repeated execution.

A.3 Use During Writing

GenAI was used as writing support in three main ways:

- Suggesting rephrasing for clarity, tone, and structure.
- Suggesting and correcting LaTeX formatting and fixing compilation or syntax errors.
- Generating first drafts of information-dense sections, especially where the text was based directly on OpenJDK source code and implementation details.

All GenAI-generated text was manually edited and verified against primary sources before inclusion in the final manuscript.

Appendix B.

Source Code

This appendix provides the key source code for the implemented optimisations. The code for the three core optimisations is from the combined (all) configuration variant, which includes all three optimisations. The code for the weak-field mechanism is from the weak_fields configuration variant. Complete sources for all variants are available in the patches/ directory of the project repository.

B.1 Dynamic-array Data Structure

File: zWeakRefArray.hpp

```
1  /*
2   * Copyright (c) 2025, Oracle and/or its affiliates. All rights
   * reserved.
3   * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4   *
5   * This code is free software; you can redistribute it and/or
   * modify it
6   * under the terms of the GNU General Public License version 2
   * only, as
7   * published by the Free Software Foundation.
8   *
9   * This code is distributed in the hope that it will be useful,
   * but WITHOUT
10  * ANY WARRANTY; without even the implied warranty of
   * MERCHANTABILITY or
11  * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
   * License
12  * version 2 for more details (a copy is included in the
   * LICENSE file that
13  * accompanied this code).
14  *
15  * You should have received a copy of the GNU General Public
   * License version
16  * 2 along with this work; if not, write to the Free Software
   * Foundation,
17  * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
```

```

18  *
19  * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
    CA 94065 USA
20  * or visit www.oracle.com if you need additional information
    or have any
21  * questions.
22  */
23
24 #ifndef SHARE_GC_Z_ZWEAKREFARRAY_HPP
25 #define SHARE_GC_Z_ZWEAKREFARRAY_HPP
26
27 #include "gc/z/zAddress.hpp"
28 #include "memory/allocation.hpp"
29 #include "utilities/debug.hpp"
30 #include "utilities/powerOfTwo.hpp"
31 #include "utilities/globalDefinitions.hpp"
32 #include <string.h>
33
34 struct ZWeakRefData {
35     zpointer* referent_field_addr;
36     zaddress* discovered_field_addr;
37     zaddress referent_addr;
38     zpointer referent_field_value;
39 };
40
41
42 class ZWeakRefArray : public AnyObj {
43 private:
44     ZWeakRefData* _data;
45     const size_t _ZWeakRefData_size = sizeof(ZWeakRefData);
46     size_t _length;
47     size_t _capacity;
48
49     void expand_to(size_t new_capacity) {
50         assert(new_capacity > _capacity, "expected growth but %ld <=
            %ld", new_capacity, _capacity);
51
52         ZWeakRefData* new_data = (ZWeakRefData*)AllocateHeap(
            new_capacity * _ZWeakRefData_size, mtGC);
53
54         if (_length > 0) {
55             // Use memcpy for trivially copyable types - much faster
            // than element-by-element copy
56             memcpy(new_data, _data, _length * _ZWeakRefData_size);
57         }
58
59         if (_data != nullptr) {
60             FreeHeap(_data);
61         }
62
63         _data = new_data;

```

```

64     _capacity = new_capacity;
65 }
66
67 void grow(size_t min_capacity) {
68     size_t new_capacity = MAX2((size_t) 8, next_power_of_2(
69         min_capacity - 1));
70     expand_to(new_capacity);
71 }
72
73 public:
74     ZWeakRefArray() : _data(nullptr), _length(0), _capacity(0) {}
75
76     ~ZWeakRefArray() {
77         if (_data != nullptr) {
78             FreeHeap(_data);
79             _data = nullptr;
80         }
81
82     void append(const ZWeakRefData& entry) {
83         if (_length >= _capacity) {
84             grow(_length + 1);
85         }
86         _data[_length] = entry;
87         _length++;
88     }
89
90     // Get entry at index
91     const ZWeakRefData& at(size_t index) const {
92         assert(index < _length, "index out of bounds: %ld (length: %
93             ld)", index, _length);
94         return _data[index];
95     }
96
97     // Current length
98     size_t length() const {
99         return _length;
100     }
101
102     // Whether the array is empty
103     bool is_empty() const {
104         return _length == 0;
105     }
106
107     // Current capacity
108     size_t capacity() const {
109         return _capacity;
110     }
111
112     void clear_and_reserve(size_t new_capacity) {
113         if (new_capacity == 0) {

```

```

113 // Special case for clearing to empty - free memory and
    reset
114 if (_data != nullptr) {
115     FreeHeap(_data);
116     _data = nullptr;
117 }
118 _length = 0;
119 _capacity = 0;
120 return;
121 }
122 _length = 0;
123 if (_data != nullptr) {
124     FreeHeap(_data);
125 }
126 _capacity = MAX2((size_t) 8, next_power_of_2(new_capacity -
    1));
127 _data = (ZWeakRefData*)AllocateHeap(_capacity *
    _ZWeakRefData_size, mtGC);
128 }
129
130 // Clear without deallocating
131 void clear() {
132     _length = 0;
133 }
134
135 // Reserve capacity without clearing
136 void reserve(size_t new_capacity) {
137     if (new_capacity > _capacity) {
138         grow(new_capacity);
139     }
140 }
141 };
142
143 #endif // SHARE_GC_Z_ZWEAKREFARRAY_HPP

```

B.2 Reference Processor

B.2.1 Interface

File: zReferenceProcessor.hpp

```

1 /*
2  * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All
    rights reserved.
3  * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4  *
5  * This code is free software; you can redistribute it and/or
    modify it

```

```

6  * under the terms of the GNU General Public License version 2
   * only, as
7  * published by the Free Software Foundation.
8  *
9  * This code is distributed in the hope that it will be useful,
   * but WITHOUT
10 * ANY WARRANTY; without even the implied warranty of
   * MERCHANTABILITY or
11 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
   * License
12 * version 2 for more details (a copy is included in the
   * LICENSE file that
13 * accompanied this code).
14 *
15 * You should have received a copy of the GNU General Public
   * License version
16 * 2 along with this work; if not, write to the Free Software
   * Foundation,
17 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
18 *
19 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
   * CA 94065 USA
20 * or visit www.oracle.com if you need additional information
   * or have any
21 * questions.
22 */
23
24 #ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
25 #define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
26
27 #include "gc/shared/referenceDiscoverer.hpp"
28 #include "gc/z/zAddress.hpp"
29 #include "gc/z/zWeakRefArray.hpp"
30 #include "gc/z/zValue.hpp"
31
32 class ConcurrentGCTimer;
33 class ReferencePolicy;
34 class ZWorkers;
35
36 class ZReferenceProcessor : public ReferenceDiscoverer {
37     friend class ZReferenceProcessorTask;
38
39 private:
40     static const size_t ReferenceTypeCount = REF_PHANTOM + 1;
41     typedef size_t Counters[ReferenceTypeCount];
42
43     ZWorkers* const _workers;
44     ReferencePolicy* _soft_reference_policy;
45     bool _uses_clear_all_soft_reference_policy;
46     ZPerWorker<Counters> _encountered_count;

```

```

47 ZPerWorker<Counters> _discovered_count;
48 ZPerWorker<Counters> _enqueued_count;
49 ZPerWorker<size_t> _encountered_weak_refs_without_queue_count
   ;
50 ZPerWorker<size_t> _discovered_weak_refs_without_queue_count;
51 ZPerWorker<size_t> _cleared_weak_refs_without_queue_count;
52 ZPerWorker<zaddress> _discovered_list;
53 ZContended<zaddress> _pending_list;
54 zaddress _pending_list_tail;
55 ZPerWorker<ZWeakRefArray>
   _discovered_weak_refs_without_queue_arr;
56 ZPerWorker<bool> _array_empty;
57 OopHandle _null_queue_handle;
58
59 bool is_inactive(zaddress reference, oop referent,
   ReferenceType type) const;
60 bool is_strongly_live(oop referent) const;
61 bool is_softly_live(zaddress reference, ReferenceType type)
   const;
62
63 bool should_discover(zaddress reference, ReferenceType type,
   oop referent) const;
64 bool try_make_inactive(zaddress reference, ReferenceType type
   ) const;
65 bool try_make_inactive_fast(const ZWeakRefData& data);
66
67 void discover(zaddress reference, ReferenceType type,
   zaddress referent);
68
69 void verify_empty() const;
70
71 void process_worker_discovered_list(zaddress discovered_list);
72
73 void process_worker_discovered_weak_refs_without_queue(
   ZWeakRefArray& weak_refs_without_queue);
74 void work();
75 void collect_statistics();
76
77 zaddress swap_pending_list(zaddress pending_list);
78
79 inline bool has_reference_queue(zaddress reference);
80
81 void initialize_null_queue_handle();
82 public:
83 ZReferenceProcessor(ZWorkers* workers);
84
85 void set_soft_reference_policy(bool clear_all_soft_references
   );
86 bool uses_clear_all_soft_reference_policy() const;
87

```

```

88 void reset_statistics();
89
90 virtual bool discover_reference(oop reference, ReferenceType
    type);
91 void process_references();
92 void enqueue_references();
93
94 void verify_pending_references();
95
96 void initializeResources();
97 };
98
99 #endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP

```

B.2.2 Implementation Excerpts

The following excerpts from `zReferenceProcessor.cpp` show the constructor and per-worker storage setup, the discovery path with queue-less weak-reference routing, the processing paths, and the queue sentinel initialisation.

Constructor and per-worker storage:

```

1  _soft_reference_policy(nullptr),
2  _uses_clear_all_soft_reference_policy(false),
3  _encountered_count(),
4  _discovered_count(),
5  _enqueued_count(),
6  _encountered_weak_refs_without_queue_count(),
7  _discovered_weak_refs_without_queue_count(),
8  _cleared_weak_refs_without_queue_count(),
9  _discovered_list(zaddress::null),
10 _pending_list(zaddress::null),
11 _pending_list_tail(zaddress::null),
12 _discovered_weak_refs_without_queue_arr(),
13 _array_empty(),
14 _null_queue_handle() {
15
16 _array_empty.set_all(true);
17 log_info(gc, ref) ("Reference Processor created, Config: All
    optimisations");
18 }
19
20 void ZReferenceProcessor::set_soft_reference_policy(bool
    clear_all_soft_references) {
21     static AlwaysClearPolicy always_clear_policy;
22     static LRUMaxHeapPolicy lru_max_heap_policy;
23
24     _uses_clear_all_soft_reference_policy =
        clear_all_soft_references;
25

```



```

26  if (clear_all_soft_references) {
27      _soft_reference_policy = &always_clear_policy;
28  } else {
29      _soft_reference_policy = &lru_max_heap_policy;
30  }
31
32  _soft_reference_policy->setup();
33  }
34
35  bool ZReferenceProcessor::uses_clear_all_soft_reference_policy
36      () const {
37      return _uses_clear_all_soft_reference_policy;
38  }
39
40  bool ZReferenceProcessor::is_inactive(zaddress reference, oop
41      referent, ReferenceType type) const {

```

Discovery path with queue-less routing:

```

1
2  assert (ZHeap::heap()->is_old(reference), "Must be old");
3  assert (is_null(reference_discovered(reference)), "Already
4      discovered");
5
6  // Update statistics
7  if (type == REF_WEAK && !has_reference_queue(reference)) {
8      _discovered_weak_refs_without_queue_count.get()++;
9  }
10 else {
11     _discovered_count.get()[type]++;
12 }
13
14 if (type == REF_WEAK && !has_reference_queue(reference)) {
15     zpointer* const referent_addr =
16         reference_referent_addr_non_vol(reference);
17     zaddress* const discovered_addr = reference_discovered_addr(
18         reference);
19     const zpointer referent_value = *referent_addr;
20
21     // WeakReference with null ReferenceQueue - remember for
22     // special processing
23     ZWeakRefArray& weak_refs_without_queue =
24         _discovered_weak_refs_without_queue_arr.get();
25     ZWeakRefData data = {referent_addr, discovered_addr,
26         referent, referent_value};
27     weak_refs_without_queue.append(data);
28     _array_empty.set(false);
29     reference_set_discovered(reference, reference); // mark as
30         discovered
31 } else {
32     if (type == REF_FINAL) {

```

```

27 // Mark referent (and its reachable subgraph) finalizable.
    This avoids
28 // the problem of later having to mark those objects if
    the referent is
29 // still final reachable during processing.
30 volatile zpointer* const referent_addr =
    reference_referent_addr(reference);
31 ZBarrier::mark_barrier_on_old_oop_field(referent_addr, true
    /* finalizable */);
32 }
33
34 // Add reference to discovered list
35 zaddress* const list = _discovered_list.addr();
36 reference_set_discovered(reference, *list);
37 *list = reference;
38 }
39 }
40
41 bool ZReferenceProcessor::discover_reference(oop reference_obj,
    ReferenceType type) {
42 if (!RegisterReferences) {
43 // Reference processing disabled
44 return false;
45 }
46
47 log_trace(gc, ref) ("Encountered Reference: " PTR_FORMAT " (%s
    )", p2i(reference_obj), reference_type_name(type));
48
49 const zaddress reference = to_zaddress(reference_obj);
50
51 // Update statistics
52 if (type == REF_WEAK && !has_reference_queue(reference)) {
53 _encountered_weak_refs_without_queue_count.get()++;
54 }
55 else {
56 _encountered_count.get()[type]++;
57 }
58
59 if (ZHeap::heap()->is_young(reference)) {
60 // Don't discover young references. Young gen reference
    processing is scary and therefore not supported.
61 return false;
62 }
63
64 volatile zpointer* const referent_addr =
    reference_referent_addr(reference);
65 const zaddress ref_raw_addr = ZBarrier::
    load_barrier_on_oop_field(referent_addr);
66 const oop referent = to_oop(ref_raw_addr);
67
68 if (!should_discover(reference, type, referent)) {

```

```

69     // Not discovered
70     return false;
71 }
72
73 discover(reference, type, ref_raw_addr);
74
75 // Discovered
76 return true;
77 }
78
79 void ZReferenceProcessor::process_worker_discovered_list(
    zaddress discovered_list) {
80     zaddress keep_head = zaddress::null;
81     zaddress keep_tail = zaddress::null;
82
83     // Iterate over the discovered list and unlink them as we go,
        potentially

```

Processing paths for separate discovered lists and dynamic arrays:

```

1     *((zpointer*)data.discovered_field_addr) = color_null(); //
        Mark as dropped
2     if (try_make_inactive_fast(data)) {
3         log_trace(gc, ref) ("\"Enqueued\" Weak Reference without
        Queue");
4         // Update statistics
5         _cleared_weak_refs_without_queue_count.get()++;
6     } else {
7         log_trace(gc, ref) ("Dropped Weak Reference without Queue");
8
9         dropped++;
10    }
11    SuspendibleThreadSet::yield();
12    weak_refs_without_queue.clear_and_reserve(dropped);
13 }
14
15 void ZReferenceProcessor::work() {
16     SuspendibleThreadSetJoiner sts_joiner;
17
18     ZPerWorkerIterator<zaddress> iter(&_discovered_list);
19     ZPerWorkerIterator<ZWeakRefArray> iter_weak_refs(&
        _discovered_weak_refs_without_queue_arr);
20     ZPerWorkerIterator<bool> iter_array_empty(&_array_empty);
21
22     zaddress* list_addr = nullptr;
23     ZWeakRefArray* array_addr = nullptr;
24     bool* array_empty = nullptr;
25
26     for (; iter.next(&list_addr) && iter_weak_refs.next(&
        array_addr) && iter_array_empty.next(&array_empty);) {
27

```

```

28     const address discovered_list = AtomicAccess::xchg(
29         list_addr, zaddress::null);
30     const bool has_array = !AtomicAccess::xchg(array_empty, true
31         );
32     const bool has_discovered = discovered_list != zaddress::
33         null;
34
35     if (has_array) {
36         // Process discovered weak references without queue
37         process_worker_discovered_weak_refs_without_queue(*
38             array_addr);
39     }
40     if (has_discovered) {
41         // Process discovered references
42         process_worker_discovered_list(discovered_list);
43     }
44 }
45
46 void ZReferenceProcessor::verify_empty() const {
47     #ifndef ASSERT
48     ZPerWorkerConstIterator<zaddress> iter(&_discovered_list);
49     for (const zaddress* list; iter.next(&list);) {
50         assert(is_null(*list), "Discovered list not empty");
51     }
52
53     ZPerWorkerConstIterator<ZWeakRefArray> iter_weak_refs(&
54         _discovered_weak_refs_without_queue_arr);
55     for (const ZWeakRefArray* array; iter_weak_refs.next(&array)
56         ;) {
57         assert(array->is_empty(), "Discovered weak refs without
58             queue not empty");
59     }
60
61     assert(is_null(_pending_list.get()), "Pending list not empty"
62         );
63 #endif
64 }

```

Queue sentinel initialisation and queue checks:

```

1     return true;
2 }
3
4     const address ref_queue_addr_value = ZBarrier::
5         load_barrier_on_oop_field(reference_queue_addr(reference));
6     const oop ref_queue = to_oop(ref_queue_addr_value);
7     bool result = ref_queue != _null_queue_handle.resolve();
8     return result;
9 }
10 void ZReferenceProcessor::initialize_null_queue_handle() {

```

```

11 EXCEPTION_MARK;
12 TempNewSymbol class_name = SymbolTable::new_symbol("java/lang
   /ref/ReferenceQueue");
13 Klass* k = SystemDictionary::resolve_or_fail(class_name, true,
   CHECK);
14 InstanceKlass* ik = InstanceKlass::cast(k);
15 ik->initialize(CHECK);
16 fieldDescriptor fd;
17 bool found = ik->find_local_field(SymbolTable::new_symbol("
   NULL_QUEUE"),
18                                     vmSymbols::referencequeue_signature
   (), &fd);
19 assert(found && fd.is_static(), "ReferenceQueue.NULL_QUEUE
   missing");
20 oop null_q = ik->java_mirror()->obj_field(fd.offset());
21 _null_queue_handle = OopHandle(Universe::vm_global(), null_q);
22 }
23
24 void ZReferenceProcessor::initializeResources() {
25   Thread* const current = Thread::current();
26   guarantee(current->is_Java_thread(), "Must be called by a
   JavaThread");
27   guarantee(!current->is_ConcurrentGC_thread(), "Must not be
   called by a GC thread");
28
29   initialize_null_queue_handle();
30 }

```

B.3 Weak Fields

B.3.1 Annotation

File: java/lang/ref/weak.java

```

1  /*
2   * Copyright (c) 2026, Oracle and/or its affiliates. All rights
   reserved.
3   * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4
5   *
6   * This code is free software; you can redistribute it and/or
   modify it
7   * under the terms of the GNU General Public License version 2
   only, as
8   * published by the Free Software Foundation. Oracle designates
   this
9   * particular file as subject to the "Classpath" exception as
   provided
   * by Oracle in the LICENSE file that accompanied this code.

```

```

10  *
11  * This code is distributed in the hope that it will be useful,
    * but WITHOUT
12  * ANY WARRANTY; without even the implied warranty of
    * MERCHANTABILITY or
13  * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
    * License
14  * version 2 for more details (a copy is included in the
    * LICENSE file that
15  * accompanied this code).
16  *
17  * You should have received a copy of the GNU General Public
    * License version
18  * 2 along with this work; if not, write to the Free Software
    * Foundation,
19  * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
20  *
21  * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
    * CA 94065 USA
22  * or visit www.oracle.com if you need additional information
    * or have any
23  * questions.
24  */
25
26 package java.lang.ref;
27
28 import java.lang.annotation.*;
29
30 /**
31  * A field annotation for references that should be treated as
    * weak by the VM.
32  */
33 @Retention(RetentionPolicy.RUNTIME)
34 @Target(ElementType.FIELD)
35 public @interface weak {}

```

B.3.2 Data Structure

File: zWeakFieldArray.hpp

```

1  /*
2  * Copyright (c) 2025, Oracle and/or its affiliates. All rights
    * reserved.
3  * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4  *
5  * This code is free software; you can redistribute it and/or
    * modify it

```

```

6  * under the terms of the GNU General Public License version 2
   * only, as
7  * published by the Free Software Foundation.
8  *
9  * This code is distributed in the hope that it will be useful,
   * but WITHOUT
10 * ANY WARRANTY; without even the implied warranty of
   * MERCHANTABILITY or
11 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
   * License
12 * version 2 for more details (a copy is included in the
   * LICENSE file that
13 * accompanied this code).
14 *
15 * You should have received a copy of the GNU General Public
   * License version
16 * 2 along with this work; if not, write to the Free Software
   * Foundation,
17 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
18 *
19 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
   * CA 94065 USA
20 * or visit www.oracle.com if you need additional information
   * or have any
21 * questions.
22 */
23
24 #ifndef SHARE_GC_Z_ZWEAKFIELDARRAY_HPP
25 #define SHARE_GC_Z_ZWEAKFIELDARRAY_HPP
26
27 #include "gc/z/zAddress.hpp"
28 #include "memory/allocation.hpp"
29 #include "utilities/debug.hpp"
30 #include "utilities/powerOfTwo.hpp"
31 #include "utilities/globalDefinitions.hpp"
32 #include <string.h>
33
34 struct ZWeakFieldData {
35     volatile zpointer* field_addr;
36     zpointer field_value;
37 };
38
39 // High-performance growable array specifically for storing
   * discovered reference data.
40 // Uses array-of-structs (AoS) layout for better cache locality
   * when processing sequentially.
41 // The stored entry type is configured by the template parameter
   * .
42 //
43 // Performance optimizations:

```

```

44 // - Uses memcpy for bulk copying instead of element-by-element
    loops
45 // - Does not zero newly allocated memory (caller responsible
    for initialization)
46 // - Single clear_and_reserve operation to avoid separate clear
    /reserve calls
47 // - Array-of-structs layout improves cache efficiency for
    sequential processing
48 class ZWeakFieldArray : public AnyObj {
49 private:
50     ZWeakFieldData* _data;
51     const size_t _ZWeakFieldData_size = sizeof(ZWeakFieldData);
52     size_t _length;
53     size_t _capacity;
54
55     void expand_to(size_t new_capacity) {
56         assert(new_capacity > _capacity, "expected growth but %ld <=
            %ld", new_capacity, _capacity);
57
58         ZWeakFieldData* new_data = (ZWeakFieldData*)AllocateHeap(
            new_capacity * _ZWeakFieldData_size, mtGC);
59
60         if (_length > 0) {
61             // Use memcpy for trivially copyable types - much faster
            than element-by-element copy
62             memcpy(new_data, _data, _length * _ZWeakFieldData_size);
63         }
64
65         if (_data != nullptr) {
66             FreeHeap(_data);
67         }
68
69         _data = new_data;
70         _capacity = new_capacity;
71     }
72
73     void grow(size_t min_capacity) {
74         size_t new_capacity = MAX2((size_t) 8, next_power_of_2(
            min_capacity - 1));
75         expand_to(new_capacity);
76     }
77
78 public:
79     ZWeakFieldArray() : _data(nullptr), _length(0), _capacity(0)
        {}
80
81     ~ZWeakFieldArray() {
82         if (_data != nullptr) {
83             FreeHeap(_data);
84             _data = nullptr;
85         }

```



```

86     }
87
88     void append(const ZWeakFieldData& entry) {
89         if (_length >= _capacity) {
90             grow(_length + 1);
91         }
92         _data[_length] = entry;
93         _length++;
94     }
95
96     // Get entry at index
97     const ZWeakFieldData& at(size_t index) const {
98         assert(index < _length, "index out of bounds: %ld (length: %
99             ld)", index, _length);
100         return _data[index];
101     }
102
103     // Current length
104     size_t length() const {
105         return _length;
106     }
107
108     // Whether the array is empty
109     bool is_empty() const {
110         return _length == 0;
111     }
112
113     // Current capacity
114     size_t capacity() const {
115         return _capacity;
116     }
117
118     void clear_and_reserve(size_t new_capacity) {
119         if (new_capacity == 0) {
120             // Special case for clearing to empty - free memory and
121             // reset
122             if (_data != nullptr) {
123                 FreeHeap(_data);
124                 _data = nullptr;
125             }
126             _length = 0;
127             _capacity = 0;
128             return;
129         }
130         _length = 0;
131         if (_data != nullptr) {
132             FreeHeap(_data);
133         }
134         _capacity = MAX2((size_t) 8, next_power_of_2(new_capacity -
135             1));

```

```

133     _data = (ZWeakFieldData*)AllocateHeap(_capacity *
134         _ZWeakFieldData_size, mtGC);
135 }
136
137 // Clear without deallocating
138 void clear() {
139     _length = 0;
140 }
141
142 // Reserve capacity without clearing
143 void reserve(size_t new_capacity) {
144     if (new_capacity > _capacity) {
145         grow(new_capacity);
146     }
147 };
148
149 #endif // SHARE_GC_Z_ZWEAKFIELDARRAY_HPP

```

B.3.3 Reference Processor

File: zReferenceProcessor.hpp

```

1  /*
2  * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All
3  * rights reserved.
4  *
5  * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
6  *
7  * This code is free software; you can redistribute it and/or
8  * modify it
9  * under the terms of the GNU General Public License version 2
10 * only, as
11 * published by the Free Software Foundation.
12 *
13 * This code is distributed in the hope that it will be useful,
14 * but WITHOUT
15 * ANY WARRANTY; without even the implied warranty of
16 * MERCHANTABILITY or
17 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
18 * License
19 * version 2 for more details (a copy is included in the
20 * LICENSE file that
21 * accompanied this code).
22 *
23 * You should have received a copy of the GNU General Public
24 * License version
25 * 2 along with this work; if not, write to the Free Software
26 * Foundation,

```

```

17  * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
18
19  * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
    CA 94065 USA
20  * or visit www.oracle.com if you need additional information
    or have any
21  * questions.
22  */
23
24  #ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
25  #define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
26
27  #include "gc/shared/referenceDiscoverer.hpp"
28  #include "gc/z/zAddress.hpp"
29  #include "gc/z/zWeakFieldArray.hpp"
30  #include "gc/z/zValue.hpp"
31
32  class ConcurrentGCTimer;
33  class ReferencePolicy;
34  class ZWorkers;
35
36  class ZReferenceProcessor : public ReferenceDiscoverer {
37  friend class ZReferenceProcessorTask;
38
39 private:
40     static const size_t ReferenceTypeCount = REF_PHANTOM + 1;
41     typedef size_t Counters[ReferenceTypeCount];
42
43     ZWorkers* const _workers;
44     ReferencePolicy* _soft_reference_policy;
45     bool _uses_clear_all_soft_reference_policy;
46     ZPerWorker<Counters> _encountered_count;
47     ZPerWorker<Counters> _discovered_count;
48     ZPerWorker<Counters> _enqueued_count;
49     ZPerWorker<size_t> _encountered_weak_fields_count;
50     ZPerWorker<size_t> _discovered_weak_fields_count;
51     ZPerWorker<size_t> _cleared_weak_fields_count;
52     ZPerWorker<zaddress> _discovered_list;
53     ZContended<zaddress> _pending_list;
54     zaddress _pending_list_tail;
55     ZPerWorker<ZWeakFieldArray> _discovered_weak_fields;
56     ZPerWorker<bool> _discovered_weak_fields_empty;
57
58     bool is_inactive(zaddress reference, oop referent,
        ReferenceType type) const;
59     bool is_strongly_live(oop referent) const;
60     bool is_softly_live(zaddress reference, ReferenceType type)
        const;
61

```

```

62  bool should_discover(zaddress reference, ReferenceType type)
        const;
63  bool should_discover_weak_field(zpointer field_value, volatile
        zpointer* field_addr) const;
64  bool try_make_inactive(zaddress reference, ReferenceType type
        ) const;
65  bool try_make_inactive_fast(const ZWeakFieldData& data);
66
67  void discover(zaddress reference, ReferenceType type);
68
69  void verify_empty() const;
70
71  void process_worker_discovered_list(zaddress discovered_list);
72
73  void process_worker_discovered_weak_fields(ZWeakFieldArray&
        weak_fields);
74  void work();
75  void collect_statistics();
76
77  zaddress swap_pending_list(zaddress pending_list);
78 public:
79     ZReferenceProcessor(ZWorkers* workers);
80
81     bool discover_weak_field(volatile zpointer* field_addr);
82
83     void set_soft_reference_policy(bool clear_all_soft_references
        );
84     bool uses_clear_all_soft_reference_policy() const;
85
86     void reset_statistics();
87
88     virtual bool discover_reference(oop reference, ReferenceType
        type);
89     void process_references();
90     void enqueue_references();
91
92     void verify_pending_references();
93 };
94
95 #endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP

```

The following excerpts from `zReferenceProcessor.cpp` show the discovery filtering, the discovery function, and the processing function.

Discovery filtering:

```

1  return true;
2  }
3
4  bool ZReferenceProcessor::should_discover_weak_field(zpointer
        referent_ptr, volatile zpointer* field_addr) const {

```

```

5  if (ZPointer::is_mark_good(referent_ptr)) {
6      // Field points to a strongly live object, no need to
        discover
7      return false;
8  }
9
10 if (is_null_any(referent_ptr)) {
11     // Field is null but not yet marked
12     AtomicAccess::cmpxchg(field_addr, referent_ptr, color_null(),
        memory_order_relaxed);
13     return false;
14 }

```

Weak-field discovery:

```

1  return false;
2  }
3
4  log_trace(gc, ref) ("Encountered Reference: " PTR_FORMAT " (%s
    )", p2i(reference_obj), reference_type_name(type));
5
6  const zaddress reference = to_zaddress(reference_obj);
7
8  // Update statistics
9  _encountered_count.get()[type]++;
10
11 if (!should_discover(reference, type)) {
12     // Not discovered
13     return false;
14 }
15
16 discover(reference, type);
17
18 // Discovered
19 return true;
20 }
21
22 bool ZReferenceProcessor::discover_weak_field(volatile zpointer*
    field_addr) {
23     if (!RegisterReferences) {
24         // Reference processing disabled
25         return false;
26     }
27
28     log_trace(gc, ref) ("Encountered Weak Reference Field: "
        PTR_FORMAT, p2i(field_addr));
29
30     // Update statistics
31     _encountered_weak_fields_count.get()++;
32
33     if (ZHeap::heap()->is_young(to_zaddress(p2i(field_addr)))) {

```

```

34 // Don't discover young references. Young gen reference
    processing is scary and therefore not supported.
35 return false;

```

Weak-field processing:

```

1  const ReferenceType type = reference_type(reference);
2  const zaddress next = reference_discovered(reference);
3  reference_set_discovered(reference, zaddress::null);
4
5  if (try_make_inactive(reference, type)) {
6      // Keep reference
7      log_trace(gc, ref) ("Enqueued Reference: " PTR_FORMAT " (%s
          ), untype(reference), reference_type_name(type));
8
9      // Update statistics
10     _enqueued_count.get() [type]++;
11
12     list_append(keep_head, keep_tail, reference);
13 } else {
14     // Drop reference
15     log_trace(gc, ref) ("Dropped Reference: " PTR_FORMAT " (%s
        ), untype(reference), reference_type_name(type));
16 }
17
18 reference = next;
19 SuspendibleThreadSet::yield();
20 }
21
22 // Prepend discovered references to internal pending list
23
24 // Anything kept on the list?
25 if (!is_null(keep_head)) {
26     const zaddress old_pending_list = AtomicAccess::xchg(
        _pending_list.addr(), keep_head);
27
28     // Concatenate the old list
29     reference_set_discovered(keep_tail, old_pending_list);
30
31     if (is_null(old_pending_list)) {
32         // Old list was empty. First to prepend to list, record
            tail
33         _pending_list_tail = keep_tail;
34     } else {
35         assert(ZHeap::heap()->is_old(old_pending_list), "Must be
            old");
36     }
37 }
38 }
39
40 void ZReferenceProcessor::process_worker_discovered_weak_fields(
    ZWeakFieldArray& weak_fields) {

```

```
41 size_t dropped = 0;
```

B.3.4 Marking Closure

The following excerpt from `zMark.cpp` shows how weak-annotated fields are detected and diverted to the weak-field discovery path during object marking.

Field annotation check and discovery diversion:

```
1  bool is_weak_annotated_field(oop* p) const {
2      if (_current_instance_klass == nullptr) {
3          return false;
4      }
5
6      const uintptr_t addr = (uintptr_t)p;
7      if (addr < _current_obj_start || addr >= _current_obj_end) {
8          return false;
9      }
10
11     const int offset = (int)(addr - _current_obj_start);
12     fieldDescriptor fd;
13     if (!_current_instance_klass->find_field_from_offset(offset,
14         false /* is_static */, &fd)) {
15         return false;
16     }
17     return fd.is_weak();
18 }
19
20 const bool _visit_metadata;
21
22 public:
23 ZMarkBarrierFollowOopClosure()
24     : OopIterateClosure(discoverer()),
25       _current_instance_klass(nullptr),
26       _current_obj_start(0),
27       _current_obj_end(0),
28       _visit_metadata(visit_metadata()) {}
29
30 void set_iteration_object(oop obj) {
31     if (obj->klass()->is_instance_klass()) {
32         _current_instance_klass = InstanceKlass::cast(obj->klass())
33             );
34         _current_obj_start = cast_from_oop<uintptr_t>(obj);
35         _current_obj_end = _current_obj_start + (obj->size() *
36             HeapWordSize);
37     } else {
38         _current_instance_klass = nullptr;
39         _current_obj_start = 0;
40         _current_obj_end = 0;
```

```

39     }
40 }
41
42 void clear_iteration_object() {
43     _current_instance_klass = nullptr;
44     _current_obj_start = 0;
45     _current_obj_end = 0;
46 }
47
48 virtual void do_oop(oop* p) {
49     if (!finalizable && is_weak_annotated_field(p)) {
50         if (ZGeneration::old()->discover_weak_field((volatile
51             zpointer*)p)) {
52             return;
53         }
54     }
55
56     switch (generation) {
57     case ZGenerationIdOptional::young:
58         ZBarrier::mark_barrier_on_young_oop_field((volatile
59             zpointer*)p);
60         break;
61     case ZGenerationIdOptional::old:
62         ZBarrier::mark_barrier_on_old_oop_field((volatile zpointer
63             *)p, finalizable);
64         break;
65     case ZGenerationIdOptional::none:
66         ZBarrier::mark_barrier_on_oop_field((volatile zpointer*)p,
67             finalizable);
68         break;
69     }
70 }

```

B.3.5 Compiler and Interpreter Support

The following excerpts show how the @weak decorator is applied in each execution tier. In all three tiers, the pattern is the same: if the field carries the weak flag and ZGC is active, ON_WEAK_OOP_REF is added to the decorator set before the access is performed.

C2 (opto/parse3.cpp) – field load and store:

```

1 void Parse::do_get_xxx(Node* obj, ciField* field, bool is_field)
2 {
3     BasicType bt = field->layout_type();
4
5     // Does this field have a constant value? If so, just push the
6     // value.
7     if (field->is_constant() &&
8         // Keep consistent with types found by ciTypeFlow: for an

```



```

7 // unloaded field type, ciTypeFlow::StateVector::
  do_getstatic()
8 // speculates the field is null. The code in the rest of
  this
9 // method does the same. We must not bypass it and use a
  non
10 // null constant here.
11 (bt != T_OBJECT || field->type()->is_loaded())) {
12 // final or stable field
13 Node* con = make_constant_from_field(field, obj);
14 if (con != nullptr) {
15     push_node(field->layout_type(), con);
16     return;
17 }
18 }
19
20 ciType* field_klass = field->type();
21 bool is_vol = field->is_volatile();
22
23 // Compute address and memory type.
24 int offset = field->offset_in_bytes();
25 const TypePtr* adr_type = C->alias_type(field)->adr_type();
26 Node *adr = basic_plus_adr(obj, obj, offset);
27 assert(C->get_alias_index(adr_type) == C->get_alias_index(
  _gvn.type(adr)->isa_ptr()),
28     "slice of address and input slice don't match");
29
30 // Build the resultant type of the load
31 const Type *type;
32
33 bool must_assert_null = false;
34
35 DecoratorSet decorators = IN_HEAP;
36 decorators |= is_vol ? MO_SEQ_CST : MO_UNORDERED;
37
38 bool is_obj = is_reference_type(bt);
39
40 if (is_obj) {
41     if (UseZGC && field->is_weak()) {
42         decorators |= ON_WEAK_OOP_REF;
43     }
44     if (!field->type()->is_loaded()) {
45         type = TypeInstPtr::BOTTOM;
46         must_assert_null = true;
47     } else if (field->is_static_constant()) {
48         // This can happen if the constant oop is non-perm.
49         ciObject* con = field->constant_value().as_object();
50         // Do not "join" in the previous type; it doesn't add
          value,
51         // and may yield a vacuous result if the field is of
          interface type.

```

```

52     if (con->is_null_object()) {
53         type = TypePtr::NULL_PTR;
54     } else {
55         type = TypeOopPtr::make_from_constant(con->isa_oopptr());
56     }
57     assert(type != nullptr, "field singleton type must be
        consistent");
58 } else {
59     type = TypeOopPtr::make_from_class(field_klass->as_klass());
60 }
61 } else {
62     type = Type::get_const_basic_type(bt);
63 }
64
65 Node* ld = access_load_at(obj, adr, adr_type, type, bt,
        decorators);
66 if (is_obj && UseZGC && field->is_weak()) {
67     insert_mem_bar(Op_MemBarCPUOrder);
68 }
69
70 // Adjust Java stack
71 if (type2size[bt] == 1)
72     push(ld);
73 else
74     push_pair(ld);
75
76 if (must_assert_null) {
77     // Do not take a trap here. It's possible that the program
78     // will never load the field's class, and will happily see
79     // null values in this field forever. Don't stumble into a
80     // trap for such a program, or we might get a long series
81     // of useless recompilations. (Or, we might load a class
82     // which should not be loaded.) If we ever see a non-null
83     // value, we will then trap and recompile. (The trap will
84     // not need to mention the class index, since the class will
85     // already have been loaded if we ever see a non-null value
86     //.)
87     // uncommon_trap(iter().get_field_signature_index());
88     if (PrintOpto && (Verbose || WizardMode)) {
89         method()->print_name(); tty->print_cr(" asserting nullness
            of field at bci: %d", bci());
90     }
91     if (C->log() != nullptr) {
92         C->log()->elem("assert_null reason='field' klass='%d'",
            C->log()->identify(field->type()));
93     }
94     // If there is going to be a trap, put it at the next
95     // bytecode:
96     set_bci(iter().next_bci());
97     null_assert(peek());

```

```

97     set_bci(iter().cur_bci()); // put it back
98 }
99 }
100
101 void Parse::do_put_xxx(Node* obj, ciField* field, bool is_field)
102 {
103     bool is_vol = field->is_volatile();
104
105     // Compute address and memory type.
106     int offset = field->offset_in_bytes();
107     const TypePtr* adr_type = C->alias_type(field)->adr_type();
108     Node* adr = basic_plus_adr(obj, obj, offset);
109     assert(C->get_alias_index(adr_type) == C->get_alias_index(
110         _gvn.type(adr)->isa_ptr()),
111         "slice of address and input slice don't match");
112     BasicType bt = field->layout_type();
113     // Value to be stored
114     Node* val = type2size[bt] == 1 ? pop() : pop_pair();
115
116     DecoratorSet decorators = IN_HEAP;
117     decorators |= is_vol ? MO_SEQ_CST : MO_UNORDERED;
118
119     bool is_obj = is_reference_type(bt);
120
121     if (is_obj) {
122         if (UseZGC && field->is_weak()) {
123             decorators |= ON_WEAK_OOP_REF;
124         }
125     }
126
127     // Store the value.
128     const Type* field_type;
129     if (!field->type()->is_loaded()) {
130         field_type = TypeInstPtr::BOTTOM;
131     } else {
132         if (is_obj) {
133             field_type = TypeOopPtr::make_from_klass(field->type()->
134                 as_klass());
135         } else {
136             field_type = Type::BOTTOM;
137         }
138     }
139     access_store_at(obj, adr, adr_type, val, field_type, bt,
140         decorators);
141
142     if (is_field) {
143         // Remember we wrote a volatile field.
144         // For not multiple copy atomic cpu (ppc64) a barrier should
145         be issued
146         // in constructors which have such stores. See do_exits() in
147         parse1.cpp.

```

```

142     if (is_vol) {
143         set_wrote_volatile(true);
144     }
145     set_wrote_fields(true);
146
147     // If the field is final, the rules of Java say we are in <
148     init> or <clinit>.
149     // If the field is @Stable, we can be in any method, but we
150     only care about
151     // constructors at this point.
152     //
153     // Note the presence of writes to final/@Stable non-static
154     fields, so that we
155     // can insert a memory barrier later on to keep the writes
156     from floating
157     // out of the constructor.
158     if (field->is_final() || field->is_stable()) {

```

C1 (c1/c1_LIRGenerator.cpp) – field load decorator:

```

1     }
2
3     DecoratorSet decorators = IN_HEAP;
4     if (is_volatile) {
5         decorators |= MO_SEQ_CST;
6     }
7     if (needs_patching) {
8         decorators |= C1_NEEDS_PATCHING;
9     } else if (UseZGC && field_type == T_OBJECT && x->field()->
10         is_weak()) {
11         decorators |= ON_WEAK_OOP_REF;
12     }
13
14     LIR_Opr result = rlock_result(x, field_type);
15     access_load_at(decorators, field_type,
16         object, LIR_OprFact::intConst(x->offset()), result,
17         info ? new CodeEmitInfo(info) : nullptr, info);

```

C1 (c1/c1_LIRGenerator.cpp) – field store decorator:

```

1
2     DecoratorSet decorators = IN_HEAP;
3     if (is_volatile) {
4         decorators |= MO_SEQ_CST;
5     }
6     if (needs_patching) {
7         decorators |= C1_NEEDS_PATCHING;
8     } else if (UseZGC && field_type == T_OBJECT && x->field()->
9         is_weak()) {
10         decorators |= ON_WEAK_OOP_REF;

```

```

10     }
11
12     access_store_at(decorators, field_type, object, LIR_OprFact::
        intConst(x->offset()),
13         value.result(), info != nullptr ? new
            CodeEmitInfo(info) : nullptr, info);
14 }
15
16 void LIRGenerator::do_StoreIndexed(StoreIndexed* x) {
17     assert(x->is_pinned(), "");
18     bool needs_range_check = x->compute_needs_range_check();

```

Template Interpreter (cpu/x86/templateTable_x86.cpp) – getfield weak path:

```

1  __ bind(notBool);
2  __ cmpl(tos_state, atos);
3  __ jcc(Assembler::notEqual, notObj);
4  // atos
5  if (UseZGC) {
6      __ testl(flags, (1 << ResolvedFieldEntry::is_weak_shift));
7      __ jcc(Assembler::zero, strongOopLoad);
8      // Weak load path
9      do_oop_load(_masm, field, rax, ON_WEAK_OOP_REF);
10     __ push(atos);
11     // Don't rewrite bytecode for weak references, the fast path
        is only for strong loads
12     __ jmp(Done);
13     __ bind(strongOopLoad);
14 }
15 do_oop_load(_masm, field, rax);
16 __ push(atos);
17 if (!is_static && rc == may_rewrite) {
18     patch_bytecode(Bytecodes::_fast_agetfield, bc, rbx);
19 }
20 __ jmp(Done);

```

Template Interpreter (cpu/x86/templateTable_x86.cpp) – putfield weak path:

```

1  // atos
2  {
3      if (UseZGC) {
4          // Test weak flag before pop(atos) overwrites flags (rax)
5          __ testl(flags, (1 << ResolvedFieldEntry::is_weak_shift));
6          __ jcc(Assembler::notZero, strongOopStore);
7          // Weak store path
8          __ pop(atos);
9          if (!is_static) pop_and_check_object(obj);
10         do_oop_store(_masm, field, rax, ON_WEAK_OOP_REF);
11         // Don't rewrite bytecode for weak references, the fast
            path is only for strong stores

```

```
12     __ jmp(Done);
13     // Strong store path
14     __ bind(strongOopStore);
15 }
16
17 __ pop(atos);
18 if (!is_static) pop_and_check_object(obj);
19 do_oop_store(_masm, field, rax);
20 if (!is_static && rc == may_rewrite) {
21     patch_bytecode(Bytecodes::_fast_aputfield, bc, rbx, true,
22                     byte_no);
23 }
24 __ jmp(Done);
25 }
```

Appendix C.

Per-Variant Patch Details

The following tables list every modified or added file in each patch variant, together with its insertion and deletion counts relative to the upstream OpenJDK master branch. New files (not present in the upstream source) are listed with their total line count and no deletion count. For variants that use the shared base patch, its files are listed first under a “Base patch” heading within each table. The `dyn_only` and `weak_fields` variants do not include the base patch.

C.1 all

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+223	–32
gc/z/zReferenceProcessor.hpp	+31	–16
gc/z/zWeakRefArray.hpp (new)	+143	—
Total	+436	–65

C.2 clear_path_dyn

File	+	–
<i>Base patch</i>		

File	+	−
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+224	−53
gc/z/zReferenceProcessor.hpp	+30	−17
gc/z/zWeakRefArray.hpp (new)	+155	—
Total	+448	−87

C.3 clear_path_only

File	+	−
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+172	−22
gc/z/zReferenceProcessor.hpp	+22	−5
Total	+233	−44

C.4 clear_path_sep

File	+	−
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0

File	+	−
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+193	−21
gc/z/zReferenceProcessor.hpp	+24	−5
Total	+256	−43

C.5 dyn_only

File	+	−
gc/z/zReferenceProcessor.cpp	+44	−28
gc/z/zReferenceProcessor.hpp	+17	−15
gc/z/zWeakRefArray.hpp (new)	+149	—
Total	+210	−43

C.6 sep_dyn

File	+	−
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+182	−27
gc/z/zReferenceProcessor.hpp	+28	−14
gc/z/zWeakRefArray.hpp (new)	+149	—
Total	+398	−58

C.7 sep_only

File	+	−
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+157	−20
gc/z/zReferenceProcessor.hpp	+17	−5
Total	+213	−42

C.8 weak_fields

File	+	−
cpu/x86/templateTable_x86.cpp	+30	−9
cpu/x86/templateTable_x86.hpp	+1	−1
cl/cl_LIRGenerator.cpp	+5	−0
cl/cl_Runtime1.cpp	+12	−3
ci/ciField.cpp	+1	−1
ci/ciField.hpp	+1	−0
ci/ciFlags.hpp	+5	−3
classfile/classFileParser.cpp	+36	−17
classfile/vmSymbols.hpp	+1	−1
gc/shared/accessBarrierSupport.cpp	+25	−6
gc/z/zBarrier.cpp	+55	−6
gc/z/zBarrier.hpp	+2	−1
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zMark.cpp	+70	−20
gc/z/zReferenceProcessor.cpp	+172	−2
gc/z/zReferenceProcessor.hpp	+20	−9
gc/z/zStat.cpp	+13	−7
gc/z/zStat.hpp	+2	−1
gc/z/zWeakFieldArray.hpp (new)	+149	—
oops/fieldInfo.hpp	+3	−0

File	+	−
oops/resolvedFieldEntry.cpp	+2	−1
oops/resolvedFieldEntry.hpp	+10	−5
opto/library_call.cpp	+216	−146
opto/parse3.cpp	+12	−0
prims/jvmtiTagMap.cpp	+41	−10
runtime/fieldDescriptor.hpp	+2	−2
services/heapDumper.cpp	+14	−6
java.base/.../ref/weak.java (new)	+35	—
Total	+940	−257

Appendix D.

Summary Statistics

This appendix provides numerical summary statistics (median, mean, and interquartile range) for all configuration variants and the key metrics discussed in Chapter 5. All tables are produced automatically by the `parse_gc_stats.py` script from the benchmark CSV data. The single-object benchmark values are per-run maxima; the multi-object benchmark values are per-run averages. Each cell is based on $n = 100$ independent benchmark runs.

D.1 Non-Strong Processing Time

D.2 Major Collection Time

D.3 Old-Generation Collection Time

D.4 Concurrent Mark Follow Time

D.5 Auxiliary GC Memory

D.6 Java Heap

D.7 Total Process RSS

Table 8.9. *Summary statistics for Non-Strong Processing across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	483.1	488.4	25.6	21.7	22.1	3.9
clear_path_only	448.6	460.1	30.1	20.6	20.6	3.7
sep_only	466.6	473.5	31.8	22.2	21.9	4.6
dyn_only	320.2	343.7	49.3	17.1	17.3	2.8
clear_path_sep	447.4	454.3	15.2	20.4	20.2	3.8
clear_path_dyn	93.0	96.9	5.0	9.3	9.1	1.8
sep_dyn	287.2	307.8	29.9	17.3	17.3	2.1
all	90.9	108.6	13.9	9.0	9.0	1.8
weak_fields	233.2	241.8	6.6	16.9	16.9	3.2

Table 8.10. *Summary statistics for Major Collection across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	3849.2	3885.6	1288.7	482.9	489.0	85.7
clear_path_only	4110.2	4296.8	1386.4	479.5	485.9	88.9
sep_only	4153.5	4276.9	1568.4	461.7	470.8	72.8
dyn_only	3950.3	4063.1	1460.6	472.5	478.7	82.6
clear_path_sep	4027.5	4057.6	1304.0	484.2	489.1	68.0
clear_path_dyn	3665.6	3808.7	1609.1	476.7	480.0	80.9
sep_dyn	4204.4	4310.8	1719.0	489.7	486.6	75.6
all	3608.0	3809.9	1625.6	467.9	470.6	76.4
weak_fields	1867.7	2019.1	190.9	341.7	340.7	54.0

Table 8.11. *Summary statistics for Old Generation across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	1861.7	1935.3	609.1	384.0	384.2	56.5
clear_path_only	1919.9	2269.9	1161.3	370.9	378.4	53.4
sep_only	1922.9	2245.7	1245.7	359.2	373.3	47.6
dyn_only	1735.3	2071.3	857.2	373.1	374.3	42.8
clear_path_sep	1871.6	2155.2	1050.4	365.3	377.9	52.9
clear_path_dyn	1545.2	1903.1	1023.5	348.4	362.5	55.9
sep_dyn	1781.5	2249.3	1362.1	375.0	380.4	54.2
all	1535.6	1794.0	960.9	357.9	361.7	54.1
weak_fields	982.3	1085.5	171.7	248.0	263.2	49.1

Table 8.12. *Summary statistics for Concurrent Mark Follow across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	1327.6	1431.0	595.8	269.8	271.5	38.3
clear_path_only	1418.7	1794.4	1159.0	269.6	269.2	34.3
sep_only	1413.6	1756.8	1258.8	257.6	264.7	29.9
dyn_only	1364.0	1711.9	868.6	264.1	267.9	31.2
clear_path_sep	1368.2	1685.2	1057.6	264.1	270.2	33.6
clear_path_dyn	1436.3	1790.7	1031.1	259.7	267.0	37.9
sep_dyn	1440.6	1926.1	1357.1	272.2	273.3	36.9
all	1419.3	1669.9	968.7	262.1	263.5	33.5
weak_fields	746.8	842.5	167.6	178.8	186.9	29.2

Table 8.13. Summary statistics for Aux Memory per-run maximum across all variants. Values in MB ($n = 100$ per variant).

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
none	73	74	2	161	161	2
clear_path_only	73	74	2	157	158	2
sep_only	73	74	2	159	160	4
dyn_only	167	168	1	164	164	4
clear_path_sep	73	74	2	157	157	1
clear_path_dyn	647	651	1	192	188	12
sep_dyn	167	168	0	165	165	4
all	455	452	1	182	181	10
weak_fields	235	237	2	131	127	11

Table 8.14. Summary statistics for Java Heap per-run maximum across all variants. Values in MB ($n = 100$ per variant).

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
none	918	919	2	1010	1011	2
clear_path_only	919	919	2	1010	1011	2
sep_only	919	919	2	1012	1011	2
dyn_only	919	919	2	1010	1011	2
clear_path_sep	918	919	2	1010	1011	2
clear_path_dyn	918	919	2	1010	1011	2
sep_dyn	920	919	2	1010	1011	2
all	918	919	2	1010	1011	2
weak_fields	460	461	2	962	962	0

Table 8.15. Summary statistics for RSS per-run maximum across all variants. Values in MB ($n = 100$ per variant).

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
none	1033	1035	6	1278	1279	7
clear_path_only	1034	1035	5	1279	1278	6
sep_only	1034	1035	5	1278	1279	6
dyn_only	1164	1164	4	1284	1285	8
clear_path_sep	1034	1035	4	1278	1279	8
clear_path_dyn	1520	1512	49	1326	1327	13
sep_dyn	1164	1164	4	1286	1287	7
all	1378	1367	33	1312	1314	11
weak_fields	739	735	12	1199	1200	10

Appendix E.

Additional Benchmark Plots

This appendix contains the remaining GC timing and memory plots from the benchmark evaluation. The metrics shown here (Old Phase: Concurrent Relocate, Young Phase: Concurrent Mark, and Young Phase: Concurrent Relocate) were not included in the main evaluation chapter as they are less central to the optimised reference processing pipeline. Memory maximum distributions are included here to complement the median percentage-difference histograms shown in the main chapter.

Additional GC timing plots are shown for Old Phase: Concurrent Relocate (Figures 8.1 and 8.2), Young Phase: Concurrent Mark (Figures 8.3 and 8.4), and Young Phase: Concurrent Relocate (Figures 8.5 and 8.6) for both benchmark families. Memory per-run maximum distributions are shown for auxiliary GC memory, Java heap, and total process RSS for both the single-object and multi-object benchmarks (Figures 8.7 and 8.8).

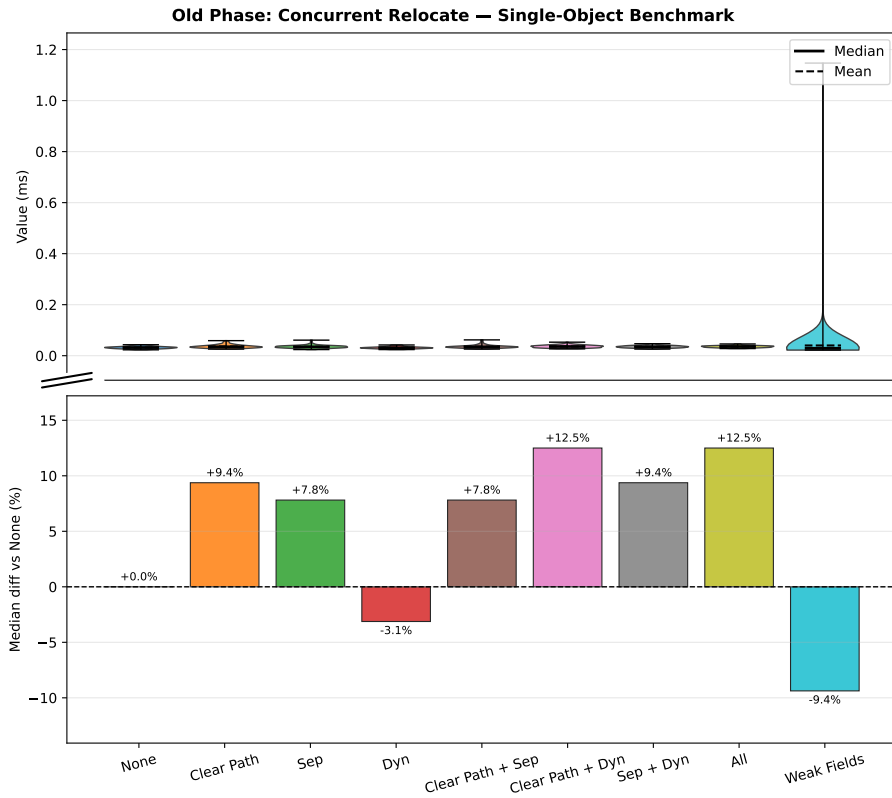


Figure 8.1. Old Phase: Concurrent Relocate timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

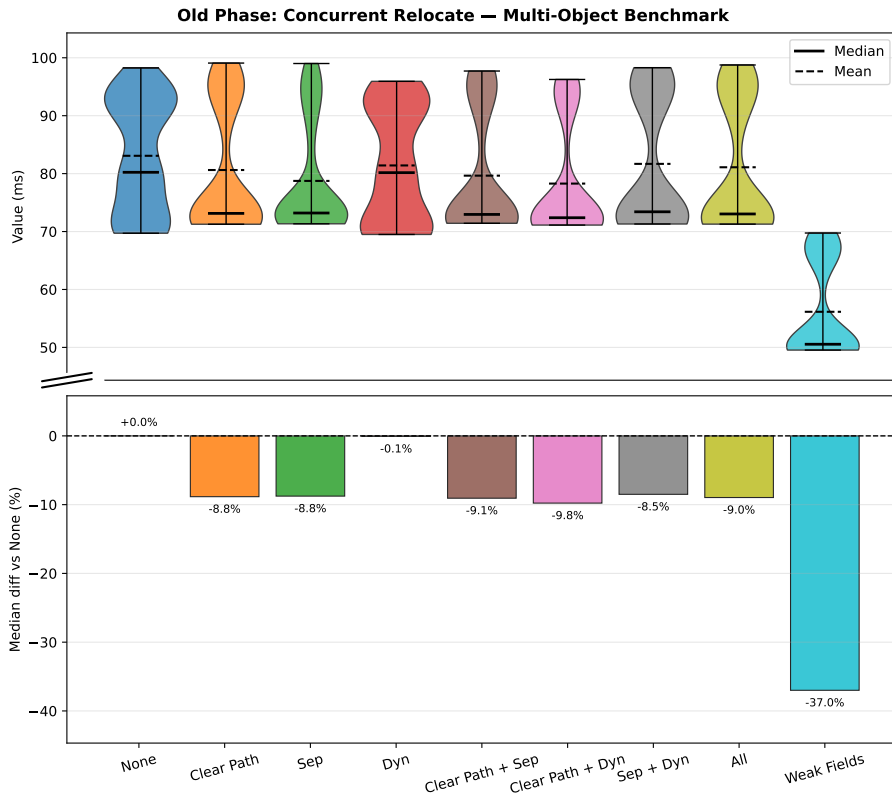


Figure 8.2. Old Phase: Concurrent Relocate timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

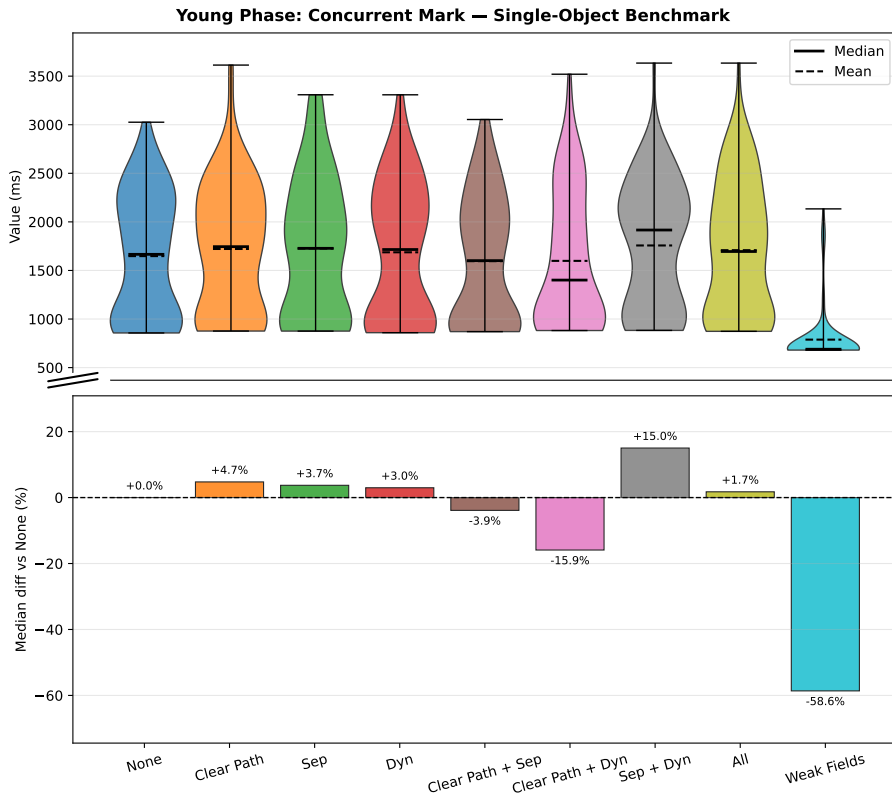


Figure 8.3. Young Phase: Concurrent Mark timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

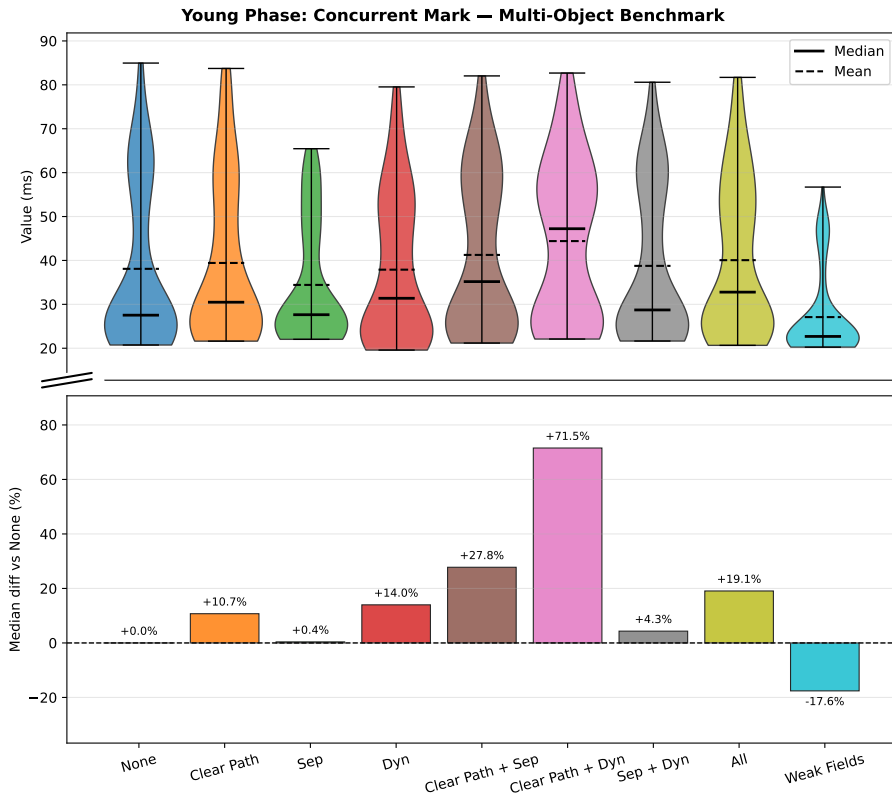


Figure 8.4. Young Phase: Concurrent Mark timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

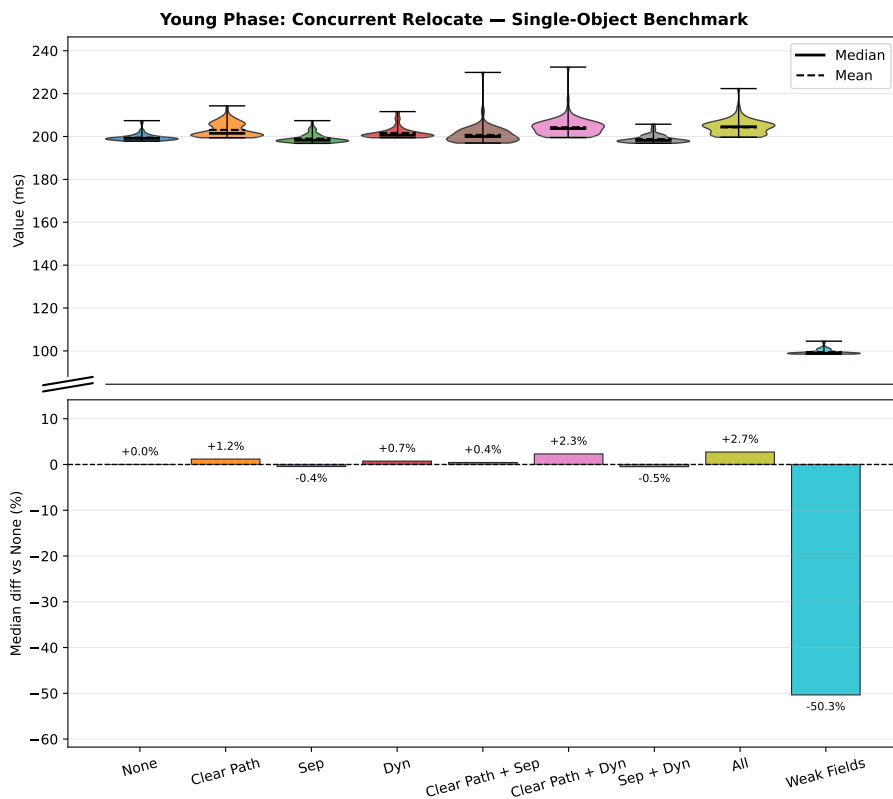


Figure 8.5. Young Phase: Concurrent Relocate timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

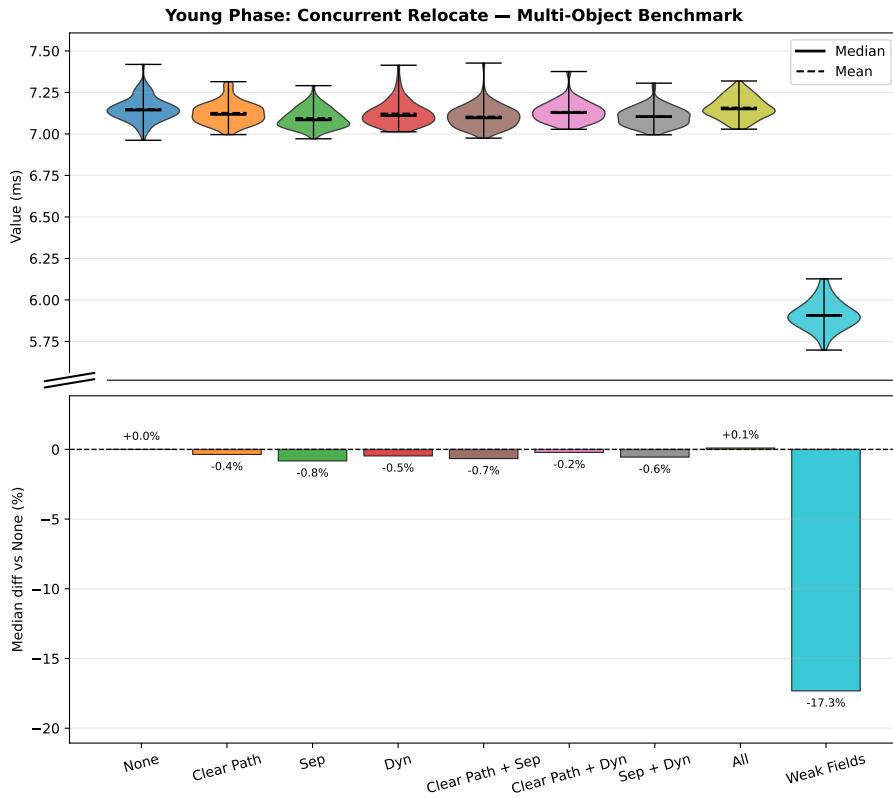


Figure 8.6. Young Phase: Concurrent Relocate timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

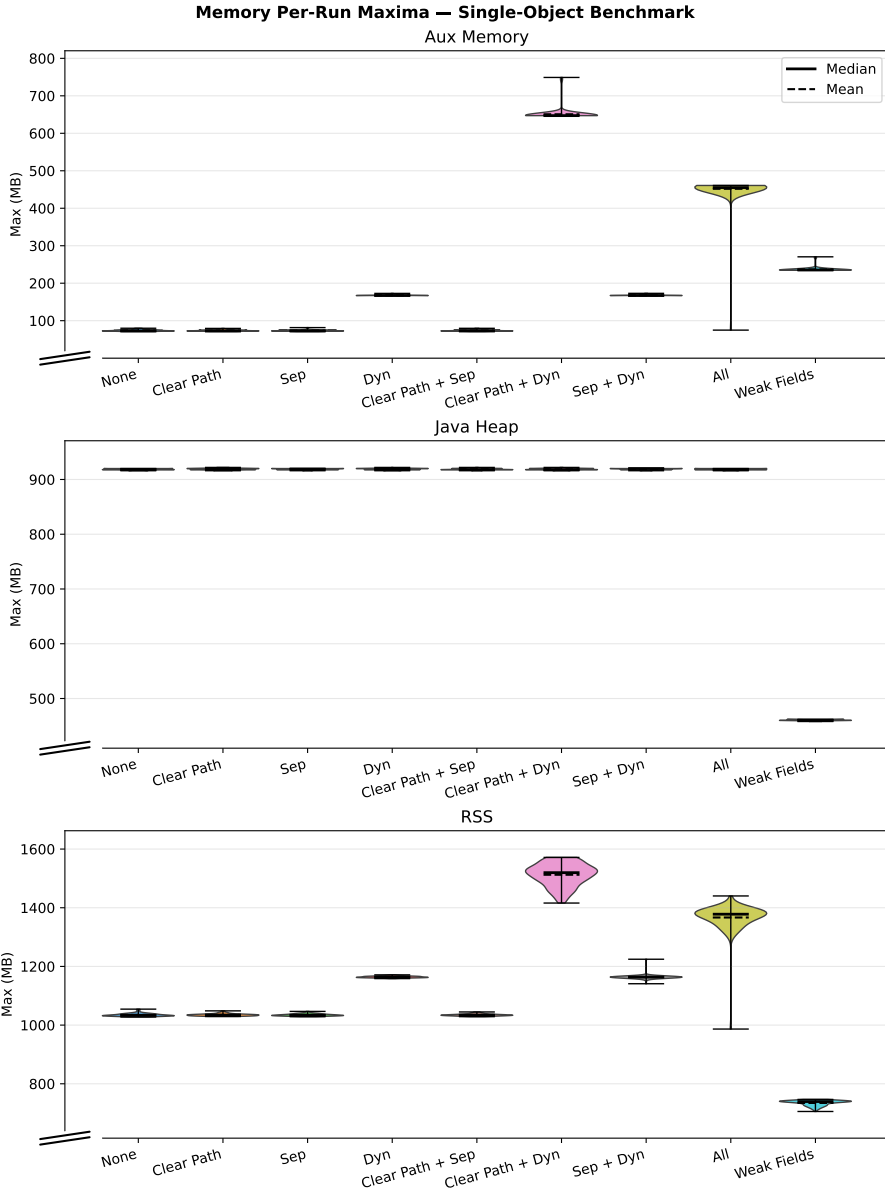


Figure 8.7. Distribution of per-run maximum memory across variants, single-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.

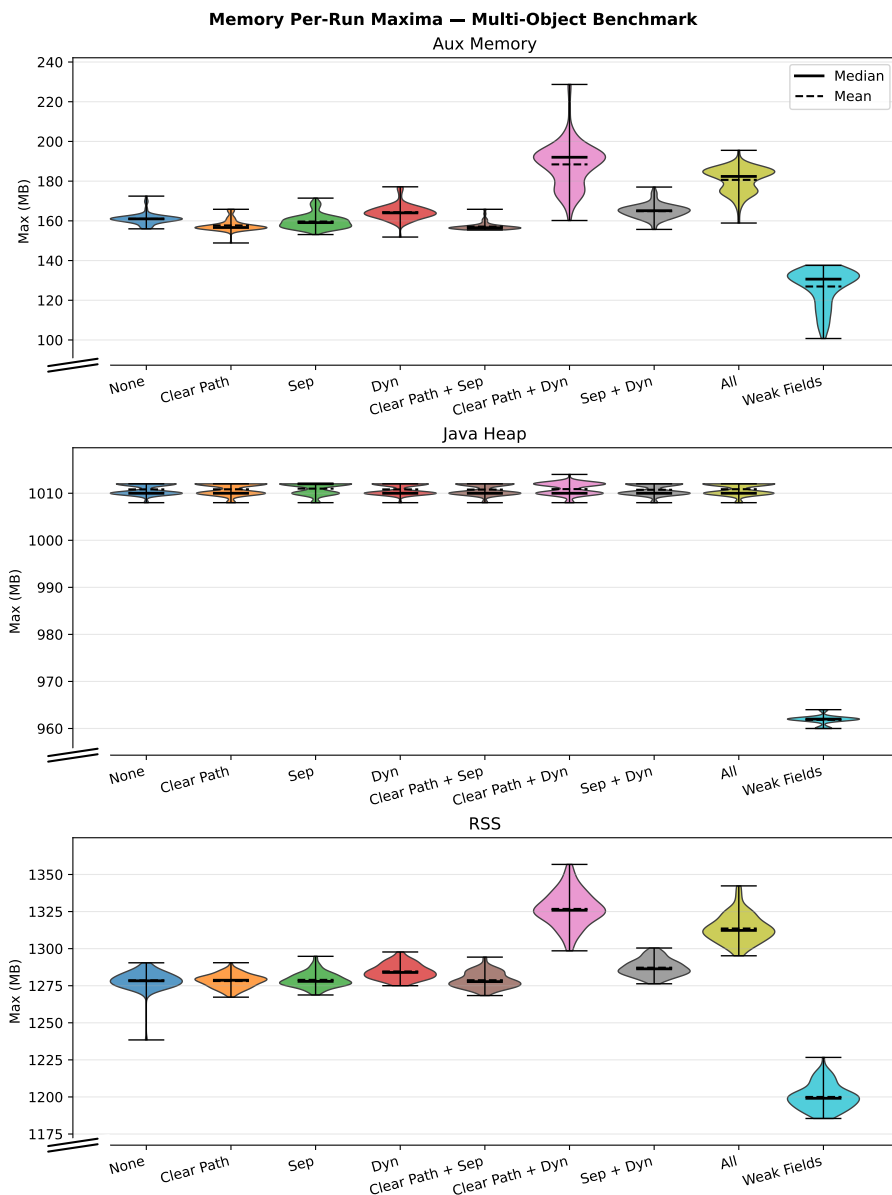


Figure 8.8. Distribution of per-run maximum memory across variants, multi-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.